life.augmented

STM32G4电机相关特别外设篇

意法半导体微控制器部门

Agenda

1 STM32G4硬件Cordic单元

2 STM32G4片内比较器

3 STM32G4片内运放

4 高性能DAC

5 滤波算法加速器（FMAC）

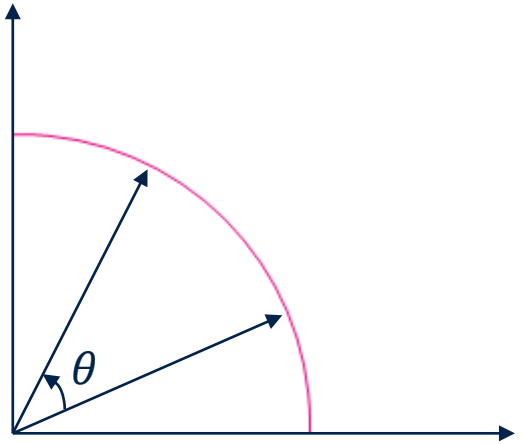
life.augmented

STM32G4硬件Cordic单元

life.augmented

Cordic简介

- CORDIC的英文全称为Coordinate Rotation Digital Computer，即坐标转换数字计算机，通过不断旋转坐标最终接近达到计算结果
- 这是“移位相加”的操作去计算各种函数的经典计算应用
- 特点是开销小，计算快，广泛应用于各种计算领域
- STM32G4集成硬核的计算单元



life.augmented

Cordic 功能

- STM32G4 Cordic 支持以下的函数

中文名	英文	表达式
余弦	Cosine	cos θ
正弦	Sine	sin θ
相位	Phase	Atan2(y,x)
取模	Modulus	$\sqrt{x^2+y^2}$
反正切	Arctangent	$\tan^{-1} x$
双曲正弦	Hyperbolic sin	sinh x
双曲余弦	Hyperbolic cosine	cosh x
双曲反正切	Hyperbolic arctangent	$\tanh^{-1} x$
自然对数	Natural logarithm	ln x
平方根	Square root	$\sqrt{x}$

数值表述

- 定点数据表述
  - CORDIC以定点有符号整型数进行运算，输入/输出值为q1.31 或者 q1.15格式
  - q1.31 格式中, 数值由1bit符号位和31bit分数构成，数值的范围为-1 (0x80000000) 到 $1 - 2^{-31}$ (0x7FFFFFFF)
  - q1.15 格式中, 数值由1bit符号位和15bit分数构成，数值的范围为-1 (0x8000) 到 $1 - 2^{-15}$ (0x7FFF)，
- 角度表述
  - 角度采用(角度/π)来表述，这样就可以更加高效的通过定点数格式来表示角度，-1代表-π， +1代表+π
- 比例系数
  - 一些函数的参数超过了定点数的表述范围，这种情况下可以对输入参数进行右移，比例系数即右移位数并保存在寄存器中。
  - 在运算前后，需要对输入值与输出值进行相应的右移与左移。

Cordic精度

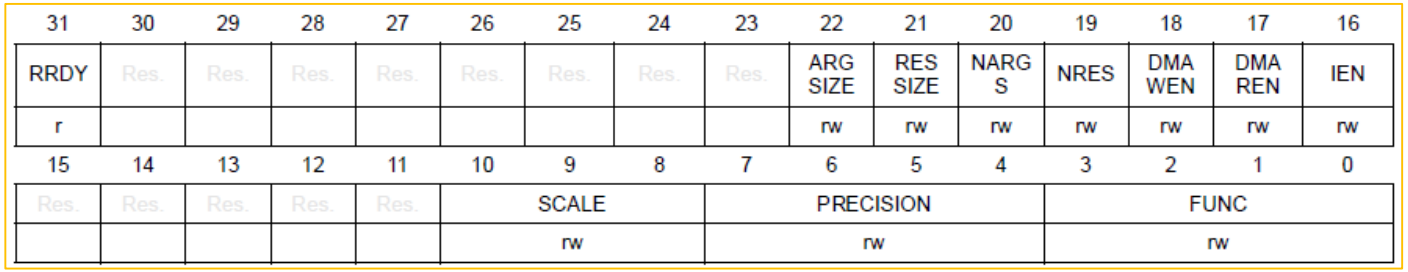
- 内部字长
  - Cordic内部采用q1.23 数据格式
- 输入/输出字长
  - 要获取最高的精度，输入与输出都需要采用q1.31格式数据
  - 如果输入采用q1.15 格式数据，不管采取何种输出，精度都被限定到q1.15
- 迭代次数(Iterations)
  - 设置Cordic时可以选定迭代次数，为4的倍数。一个时钟周期可以执行4次迭代。
  - 考虑最快的速度，在达到需要精度的情况下，选用最少的迭代次数。
  - 在达到最大精度后继续使用迭代将逐渐降低精度

精度 vs 迭代次数

Function	Number of iterations	Number of cycles	Max residual error(1)	
			q1.31 format	q1.15 format
Sin, Cos, Phase(2), Mod, Atan(4)	4	1	2^{-3}	2^{-3}
	8	2	2^{-7}	2^{-7}
	12	3	2^{-11}	2^{-11}
	16	4	2^{-15}	2^{-15}
	20	5	2^{-18}	2^{-16}
	24	6	2^{-19}	2^{-16}

寄存器说明

- Cordic三个寄存器：
  - CSR：　控制/状态寄存器
  - WDATA：输入寄存器
  - RDATA：输出寄存器



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RRDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ARG SIZE	RES SIZE	NARG S	NRES	DMA WEN	DMA REN	IEN
r									rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	SCALE				PRECISION			FUNC			
					rw				rw			rw			

Cordic_CSR

Bit位	说明
RRDY	输出结果就绪标志
ARGSIZE	输入参数数据长度，0：32-bit，1：16-bit
RESSIZE	输出结果数据长度，0：32-bit，1：16-bit
NARGS	输入参数数量
NRES	输出结果数量
DMAWEN	DMA写入参数通道使能
DMAREN	DMA读取结果数据通道使能
SCALE	比例系数
PRECISION	设置精度需求
FUNC	选择cordic函数

Cordic计算输入与输出

- 按照每个Cordic函数的设定，输入序列
- 按照每个Cordic函数的设定，读取序列

Function	Primary argument (ARG1)	Secondary argument (ARG2)	Primary result (RES1)	Secondary result (RES2)
Cosine	angle θ	modulus m	$m \cdot \cos\theta$	$m \cdot \sin\theta$
Sine	angle θ	modulus m	$m \cdot \sin\theta$	$m \cdot \cos\theta$
Phase	x	y	atan2(y,x)	$\sqrt{x^2+y^2}$
Modulus	x	y	$\sqrt{x^2+y^2}$	atan2(y,x)
Arctangent	x	none	$\tan^{-1} x$	none
Hyperbolic cosine	x	none	cosh x	sinh x
Hyperbolic sine	x	none	sinh x	cosh x
Hyperbolic arctangent	x	none	$\tanh^{-1} x$	none
Natural logarithm	x	none	ln x	none
Square root	x	none	$\sqrt{x}$	none

life.augmented

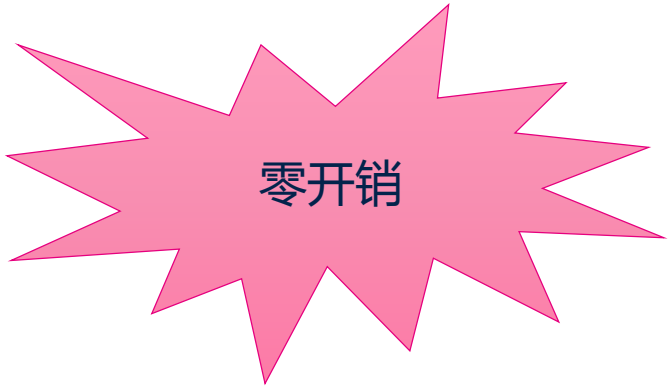
计算举例---余弦函数

Parameter	Description	Range
ARG1	Angle θ in radians, divided by π	[-1 1]
ARG2	Modulus m	[0 1]
RES1	m.cos θ	[-1 1]
RES2	m.sin θ	[-1 1]
SCALE	Not applicable	0

- 计算[-π，π]的cosine 值，可用于极坐标到直角坐标的转换。
- 第一输入参数ARG1为角度θ(弧度)，在写入ARG1前必须除以 π
- 第二输入参数ARG2为模值m，如果m大于1，必须将其调整到q1.31[0,1]的范围
- 第一输出结果RES1为m*cosθ
- 第二输出结果RES2为m*sinθ

Cordic应用模式

- 零开销单次模式(Zero-overhead single-shot mode)
  - 执行单次运算的最快方式
- 零开销流水线模式(Zero-overhead pipelined mode)
  - 执行多个连续运算的最快方式
- 查询模式
- 中断模式
- DMA 模式



Cordic vs ARM fast math

- 缓冲数据转换
  - 将保存在缓冲区中3024个角度值转换为sin值(正弦波周期1Khz，采样频率48Khz)
  - 角度值由CPU提前计算

q1.15 fixed point:
(precision = 4)

ARM fast math arm_sin_q15()	Cordic: zero-overhead mode	Cordic: DMA in/out mode
36 cycles/sample	7 cycles/sample	11 cycles/sample
-	x5 acceleration	x3 acceleration
100%CPU	100% CPU	0% CPU
Max error: 0.00012	Max error: 0.00004	Max error: 0.00004
-	x3 precision	x3 precision

q1.31 fixed point:
(precision = 6)

ARM fast math arm_sin_q31()	Cordic: zero-overhead mode	Cordic: DMA in/out mode
41 cycles/sample	8 cycles/sample	12 cycles/sample
-	x5 acceleration	x3 acceleration
100%CPU	100% CPU	0% CPU
Max error: 0.00002	Max error: 0.000002	Max error: 0.000002
-	x10 precision	x10 precision

life.augmented

电机应用中计算速度对比

- 电机矢量控制的Park变换(一次性计算)
  - 广泛应用于电机控制中:
    - ✓ X = D.cos θ - Q.sin θ
    - ✓ Y = D.sin θ + Q.cos θ

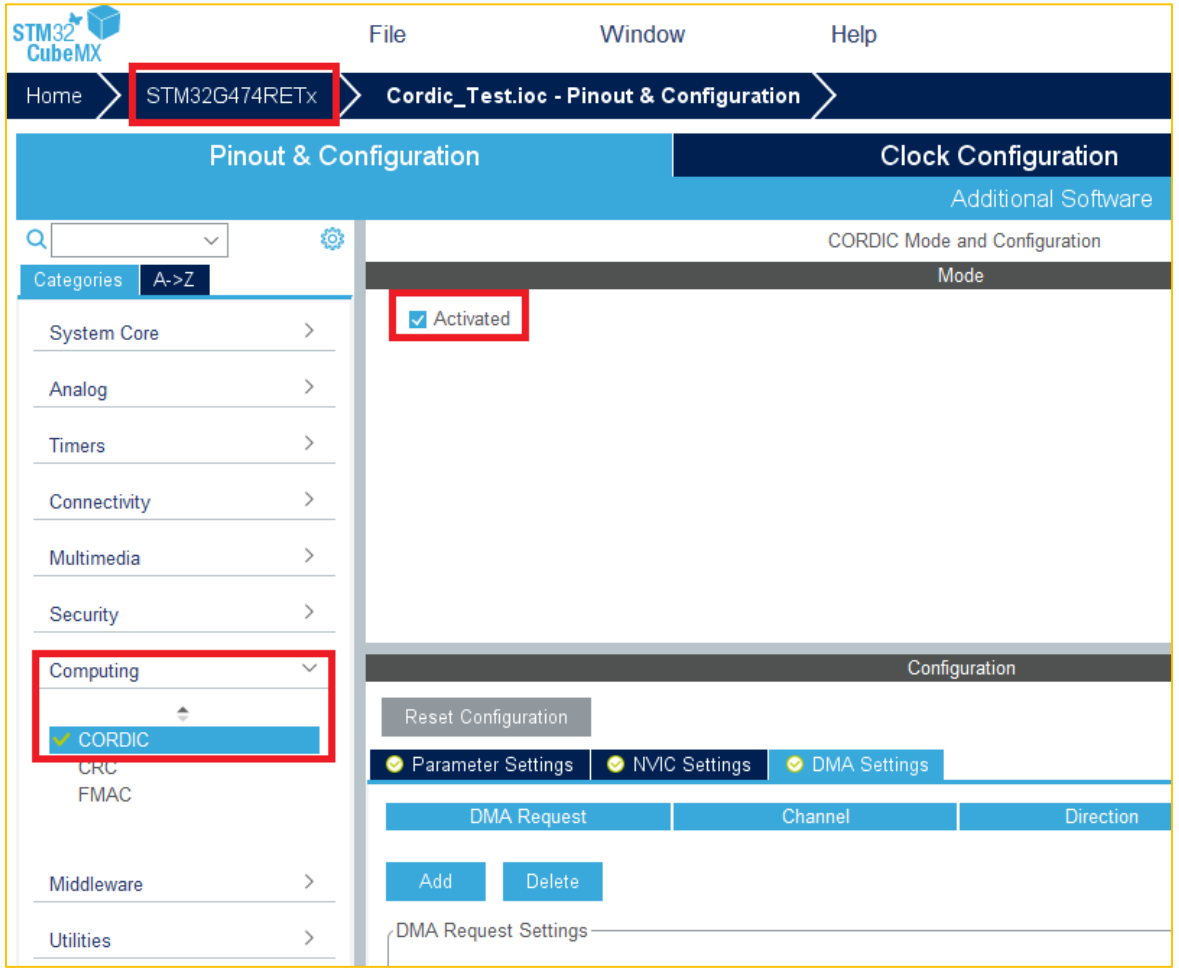
q1.15-bit fixed point:

(precision = 4)

ARM fast math	Cordic: zero-overhead mode
243 cycles	48 cycles
-	x5 acceleration

Cordic软件实现

- CubeMx使能Cordic单元



- 基本操作步骤：
  - ① CORDIC->CSR = ...;
  - ② CORDIC->WDATA = ...;
  - ③ Read_Data = CORDIC->RDATA;

life.augmented

Cordic在电机控制算法中应用

- MC SDK V5.4.3中STM32G4计算三角函数

```
__weak Trig_Components MCM_Trig_Functions( int16_t hAngle )
{

 union u32toi16x2 {
    uint32_t CordicRdata;
    Trig_Components Components;
  } CosSin;

  /* Configure CORDIC */
  WRITE_REG(CORDIC->CSR, CORDIC_CONFIG_COSINE);
  LL_CORDIC_WriteData(CORDIC, 0x7FFF0000 + (uint32_t) hAngle);
  /* Read angle */
  CosSin.CordicRdata = LL_CORDIC_ReadData(CORDIC);
  return (CosSin.Components);

}
```

第一步：配置Cordic CSR寄存器，计算Sin、Cos

第二步：写入角度值

第三步：读取Sin/Cos计算结果

life.augmented

Cordic在电机控制算法中应用

- MC SDK V5.4.3中STM32G4计算平方根

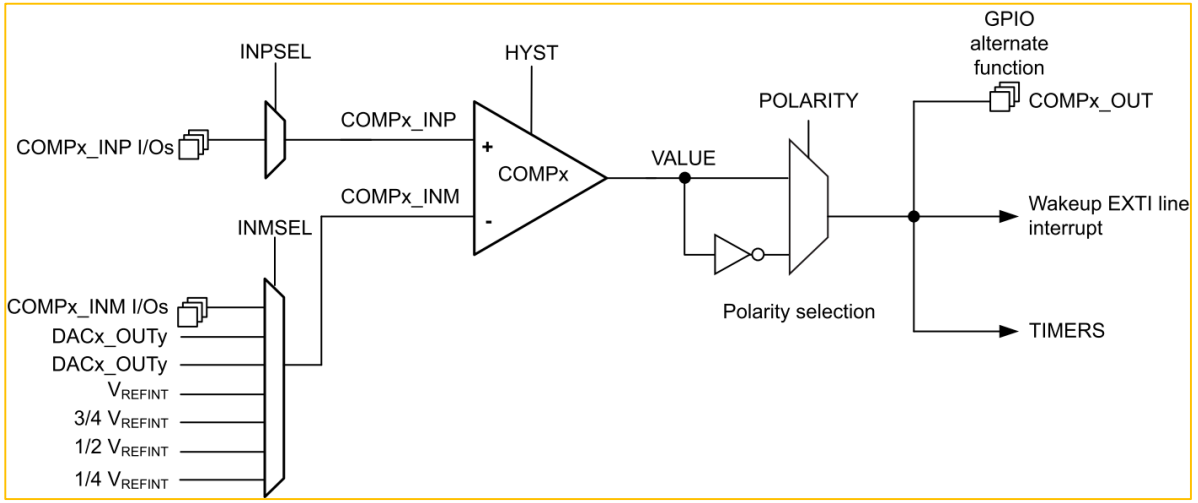
```
__weak int32_t MCM_Sqrt( int32_t wInput )
{
  int32_t wtemprootnew;

  if ( wInput > 0 )
  {
    /* Configure CORDIC */
    WRITE_REG(CORDIC->CSR, CORDIC_CONFIG_SQRT);
    LL_CORDIC_WriteData(CORDIC, (uint32_t) (wInput));
    /* Read sqrt and return */
    wtemprootnew = ((int32_t) (LL_CORDIC_ReadData(CORDIC))>>15);


  }
  else
  {
    wtemprootnew = ( int32_t )0;
  }

  return ( wtemprootnew );
}
```

STM32G4片内比较器

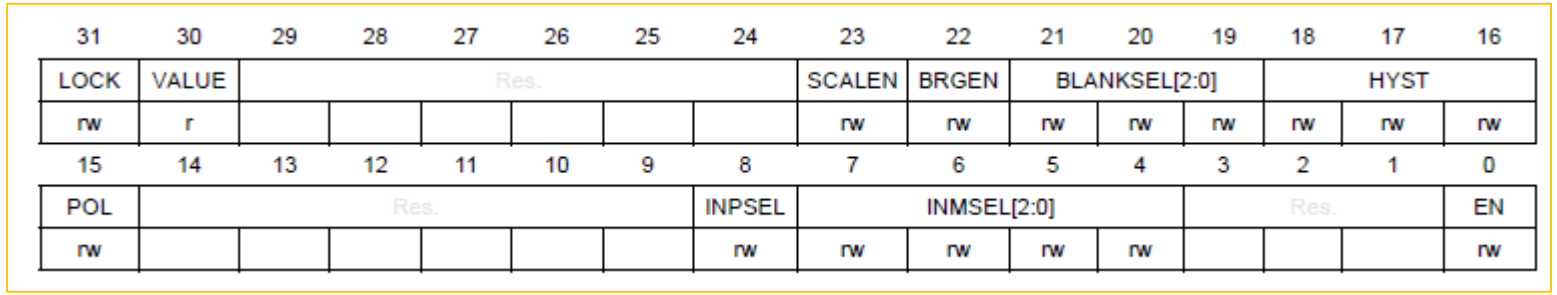
life.augmented

STM32G4内部比较器特色

- 多达7个比较器通道(Up to 7 x COMP)
- 传输延时16.7ns
- 输入:
  - 外部管脚
  - 内部信号:
    - DAC通道输出
    - Vrefint : ¼, ½, ¾, 1
- 输出:
  - 外部管脚
  - 定时器(TIMs，HRTIM)输入 (trigger, break signals, OCREF_CLR inputs)
- 输出消隐特性 – 去除干扰尖峰
- 可编程回差
- 低功耗状态唤醒 (LP Run, Sleep, LP Sleep, Stop)



寄存器说明



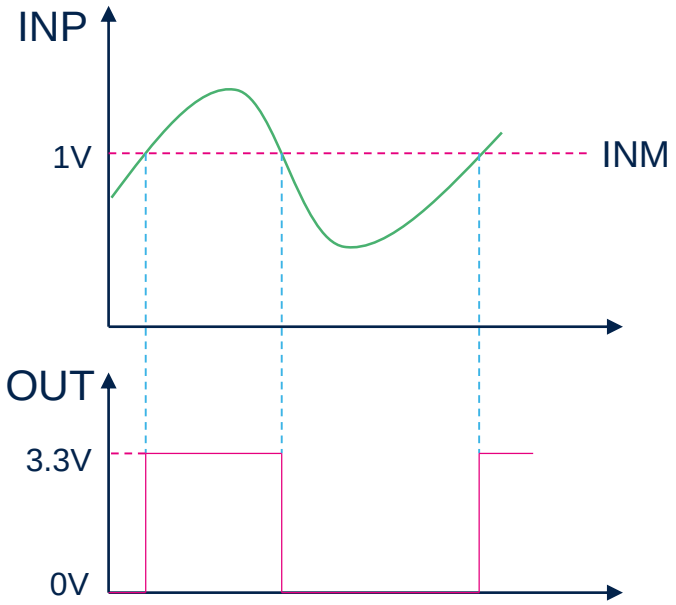
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCK	VALUE	Res.						SCALEN	BRGEN	BLANKSEL[2:0]			HYST		
rw	r							rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
POL	Res.						INPSEL	INMSEL[2:0]				Res.			EN
rw							rw	rw	rw	rw	rw				rw

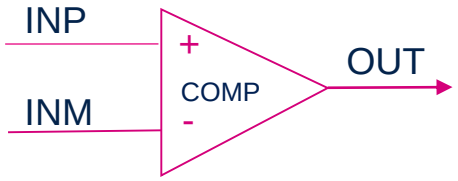
Bit位	说明
LOCK	保护位
VALUE	比较器输出状态，0或1
SCALEN	VREFINT作为比较电平的使能开关
BRGEN	VREFINT内部电阻桥使能，使用1/4,1/2,3/4时必须使能
BLANKSEL	比较器Blank信号选择
HYST	比较器回差电平配置
POL	比较器极性选择
INPSEL	比较器同向端配置，0/1分别对应不同管脚
INMSEL[2:0]	比较器反向端配置
EN	比较器使能，1-使能，0-关闭

life.augmented

比较器基本应用

- 比较器反向端接固定电平（1V）举例





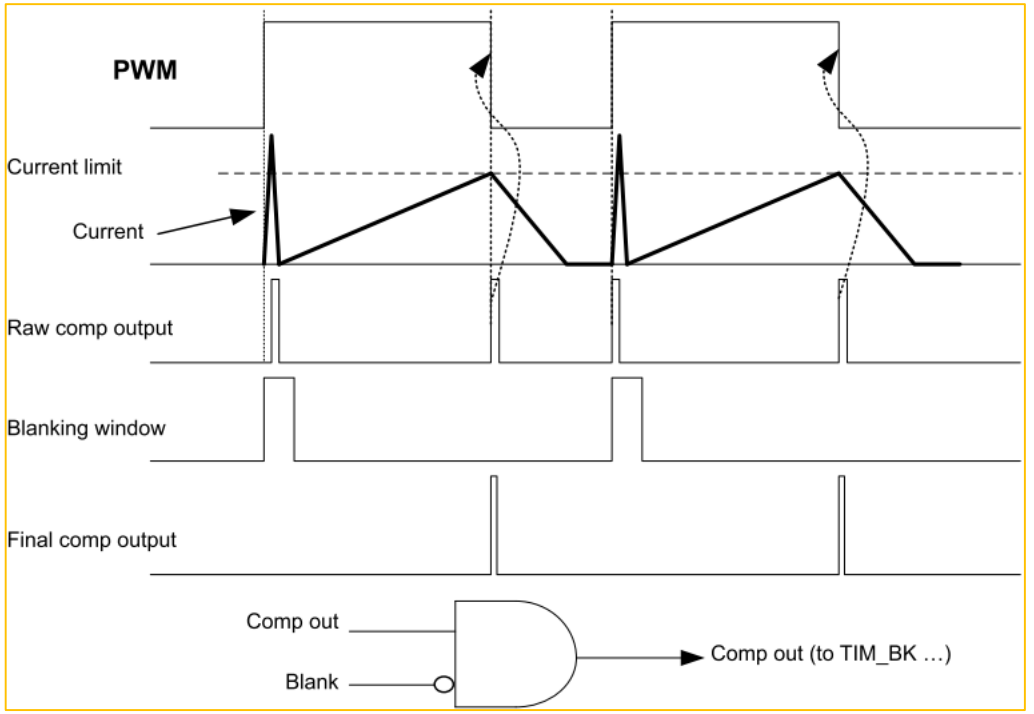
t_D(4)	Propagation delay (From COMP input pin to COMP output pin) for 200 mV step with 100 mV overdrive	50pF load on output	V_{DDA} < 2.7 V	-	-	35	ns
			V_{DDA} ≥2.7 V	-	16.7	31	ns

输出延迟时间

比较器消隐（Blank）

- 比较器消隐特征:
  - 消隐信号能够屏蔽比较器输出
    - 消隐窗口由定时器比较输出定义 (TIMx_OCy)
- 应用: 屏蔽开关动作电流尖峰

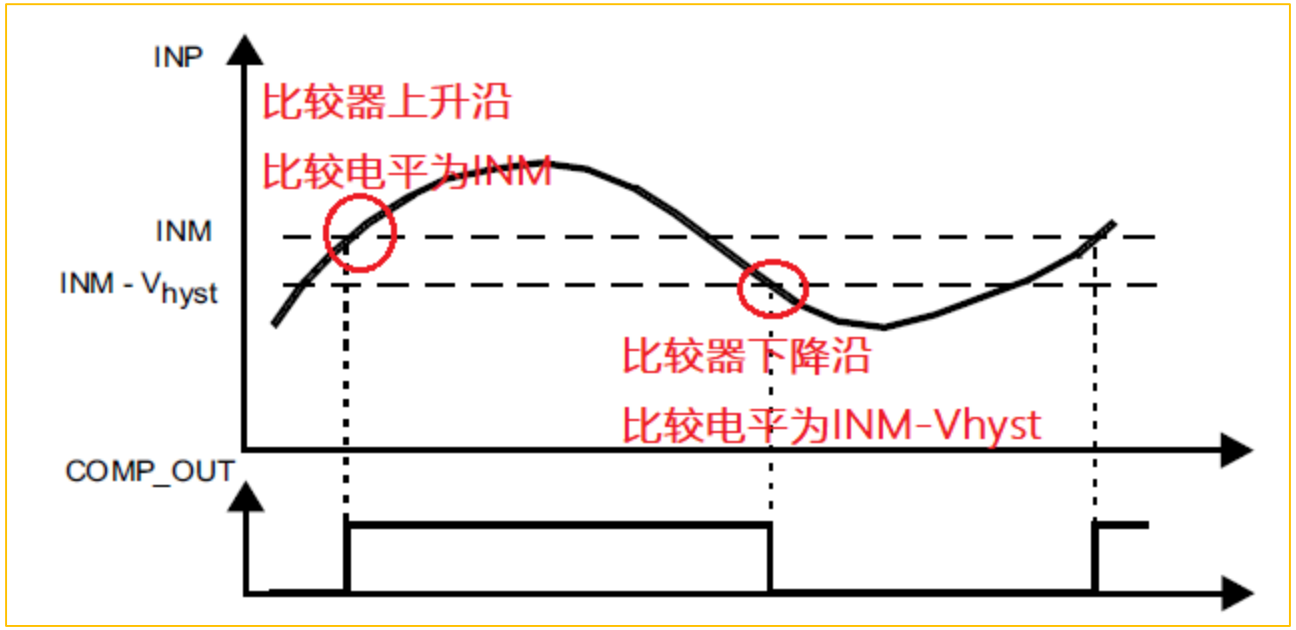
BLANKSEL [2:0]	COMP1	COMP2	COMP3	COMP4	COMP5	COMP6	COMP7
001	TIM1_OC5	TIM1_OC5	TIM1_OC5	TIM3_OC4	TIM2_OC3	TIM8_OC5	TIM1_OC5
010	TIM2_OC3	TIM2_OC3	TIM3_OC3	TIM8_OC5	TIM8_OC5	TIM2_OC4	TIM8_OC5
011	TIM3_OC3	TIM3_OC3	TIM2_OC4	TIM15_OC1	TIM3_OC3	TIM15_OC2	TIM3_OC3
100	TIM8_OC5	TIM8_OC5	TIM8_OC5	TIM1_OC5	TIM1_OC5	TIM1_OC5	TIM15_OC2
101	TIM20_OC5	TIM20_OC5	TIM20_OC5	TIM20_OC5	TIM20_OC5	TIM20_OC5	TIM20_OC5
110	TIM15_OC1	TIM15_OC1	TIM15_OC1	TIM15_OC1	TIM15_OC1	TIM15_OC1	TIM15_OC1
111	TIM4_OC3	TIM4_OC3	TIM4_OC3	TIM4_OC3	TIM4_OC3	TIM4_OC3	TIM4_OC3



life.augmented

比较器回差电压

- 比较器可编程回差:
  - 8级回差:
    - 0 – 63mV，步长为 9mV
    - 设置: HYST[2:0]
  - 回差仅影响输出下降沿

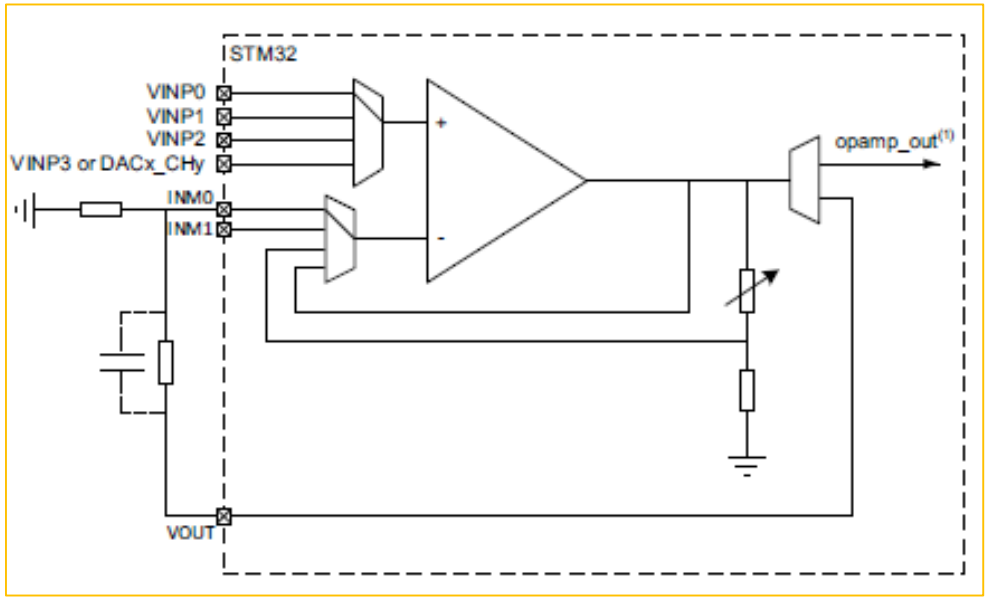


STM32G4片内运放

life.augmented

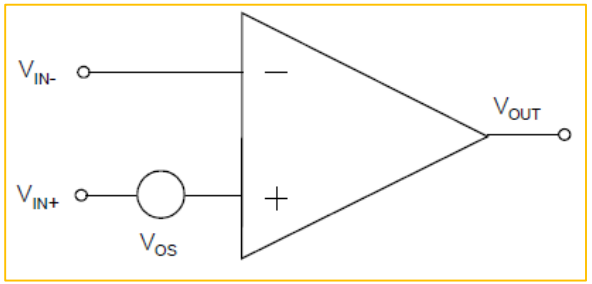
STM32G4内部运放特色

- 高增益带宽积(GBW)—13MHz
- 高压摆率(SR)—45V/us（High-speed mode）
- 低输入偏置，客户可校准
- 多达6通道运放
- 灵活的配置模式
- PGA增益可调整
  - Positive: 2, 4, 8, 16, 32, 64
  - Negative: -1, -3, -7, -15, -31, -63
- 内部输出到ADC（无需占用外部Pin）



运放参数--失调电压

- 失调电压Vos：
  - 运放开环使用时，让运放直流输出为0时在运放输入端的直流电压差
  - 25度时测试的失调电压，单位：uV或者mV
  - 因为温度影响的偏移电压数据，单位：$\mu V/℃$
- 影响：当放大信号时，输出信号会叠加失调电压的倍数



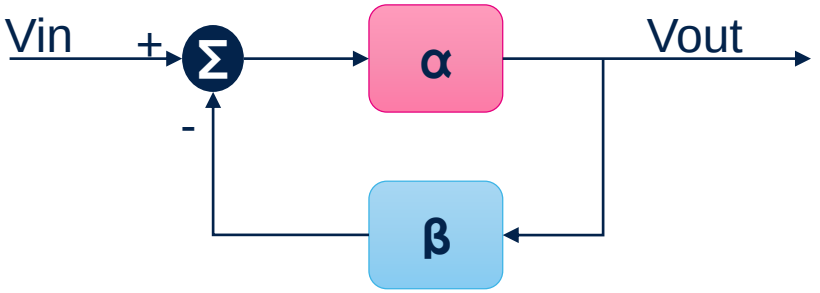
VI_{OFFSET}	Input offset voltage	25 °C, No Load on output.	-	-	±1.5	mV
		All voltage/temperature.	-	-	±3	
ΔVI_{OFFSET}	Input offset voltage drift	-	-	±10	-	µV/°C

STM32G4内部运放失调电压参数

运放参数--开环增益

- 理想运放的开环增益为无穷大，实际上都做不到，有一定数据
- 实际设计运放电路时候要计算增益误差

$$增益误差(\%) = 100 * \frac{\frac{\alpha}{1+\alpha\beta} - \frac{1}{\beta}}{\frac{1}{\beta}} = \frac{\alpha\beta}{1+\alpha\beta} - 1$$



α

AO	Open loop gain	100mV ≤ Output dynamic range ≤ VDDA - 100mV	65	95	-	dB
		200mV ≤ Output dynamic range ≤ VDDA - 200mV	75	95	-	

STM32G4内部运放开环增益参数

$$实际闭环增益 = \frac{\alpha}{1+\alpha\beta}$$

$$理想闭环增益 = \frac{1}{\beta}$$

α-开环增益

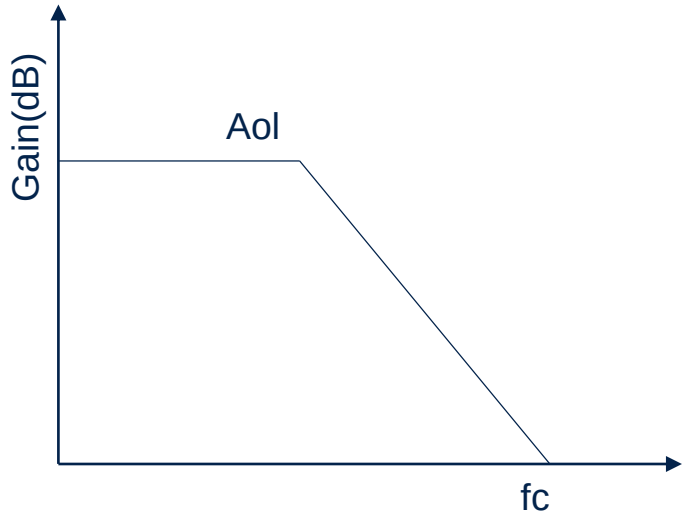
β-反馈系数

life.augmented

运放参数--增益带宽积

- 增益带宽积（GBW）：在某频率下开环电压增益与频率的乘积
- 表征的是运放交流特性

$$GBW = f * Gain$$

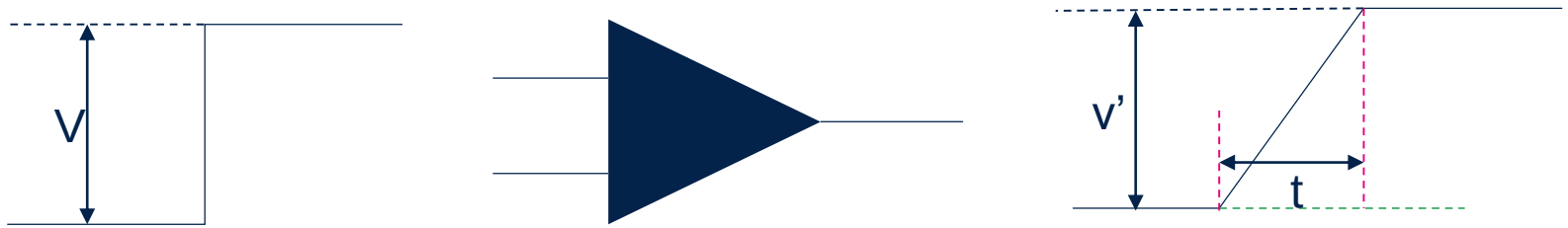


GBW	Gain Bandwidth Product	100mV ≤ Output dynamic range ≤ VDDA - 100mV	7	13	-	MHz

STM32G4内部运放增益带宽积

运放参数--压摆率

- 压摆率(SR)： 当输入运放一个阶跃信号时，运放输出信号的最大变化速率
- $SR = \frac{dv}{dt}$

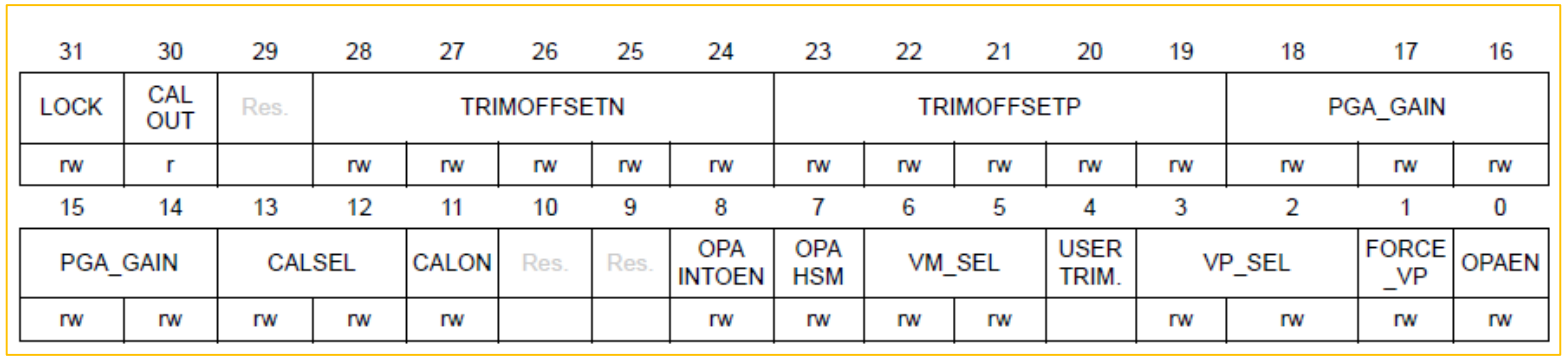


SR(3)	Slew rate (from 10 and 90% of output voltage)	Normal mode	2.5	6.5	-	V/µs
		High-speed mode	18	45	-	

STM32G4内部运放压摆率

寄存器说明

OPAMPx_CSR



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCK	CAL OUT	Res.	TRIMOFFSETN					TRIMOFFSETP					PGA_GAIN		
rw	r		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

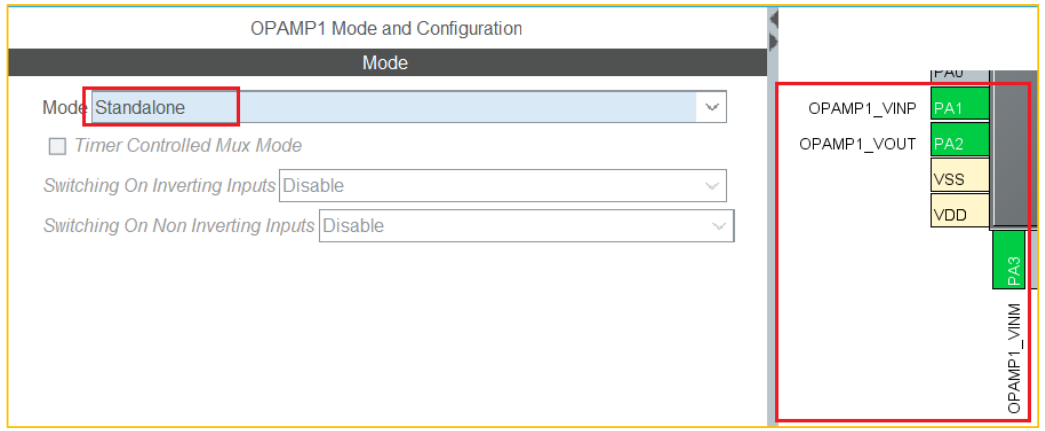
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PGA_GAIN		CALSEL		CALON	Res.	Res.	OPA INTOEN	OPA HSM	VM_SEL		USER TRIM.	VP_SEL		FORCE _VP	OPAEN
rw	rw	rw	rw	rw			rw	rw	rw	rw		rw	rw	rw	rw

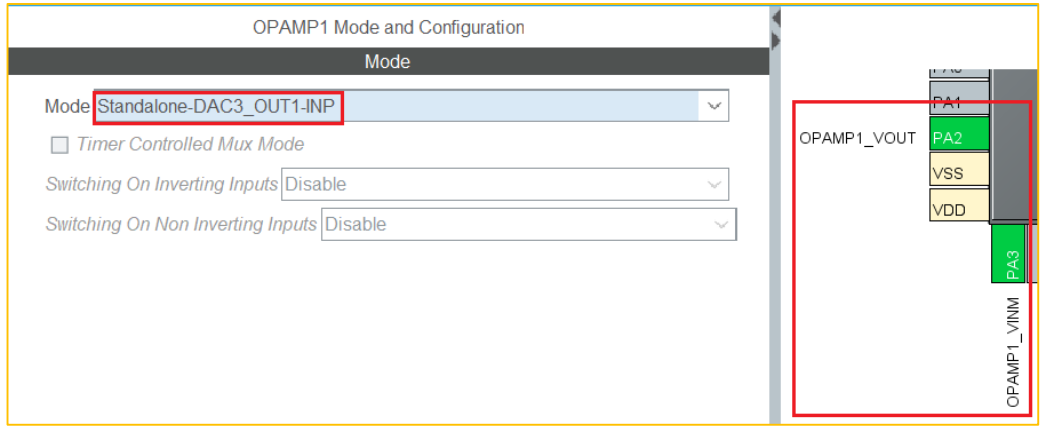
Bit位	说明
LOCK	保护位
CALOUT	校准成功只读位 1->0 校准成功
TRIMOFFSETN/P	Trim
PGA_GAIN	PGA增益设定
CALSEL	校准选择
CALON	校准使能
OPAINTOEN	运放输出配置，0-输出连接到管脚，1-内部连接
OPAHSM	运放模式选择，0-Normal，1-High speed
VM_SEL	运放反向端选择
VP_SEL	运放同向端选择
FORCE_VP	同向端连接，1-校准模式
OPAEN	运放使能，1-使能，0-关闭

life.augmented

独立模式与软件配置

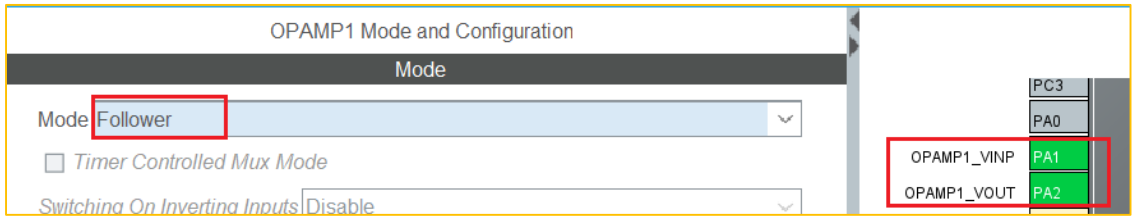
- 独立模式（Standalone）

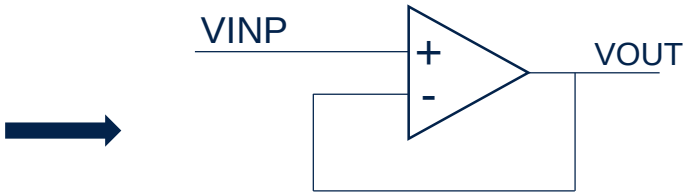


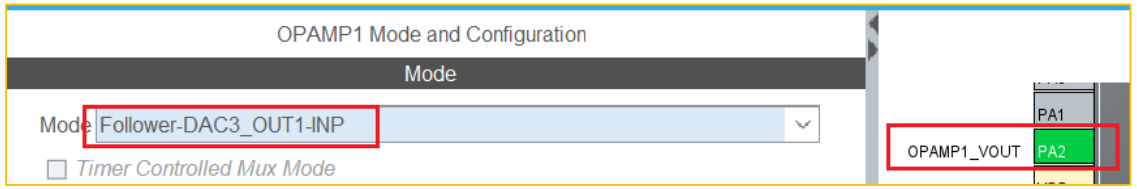


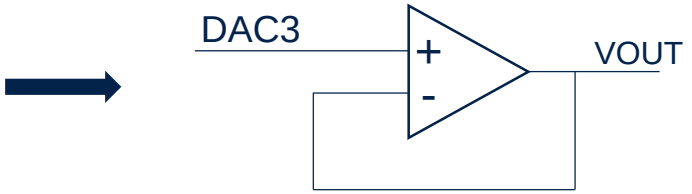
跟随模式与软件配置

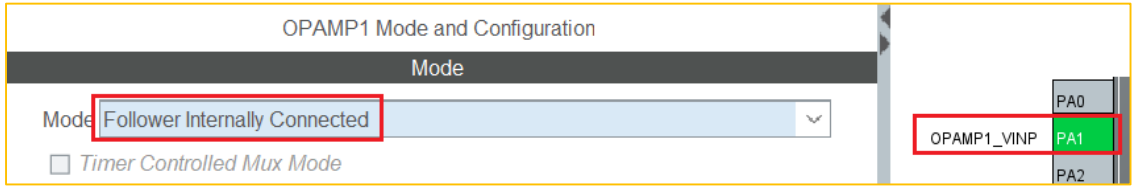
- 跟随模式（Follower）

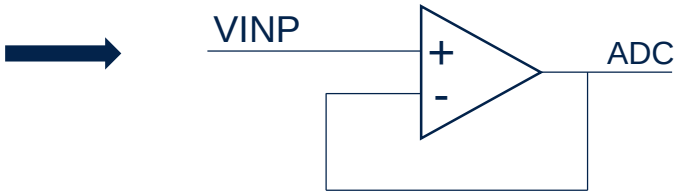








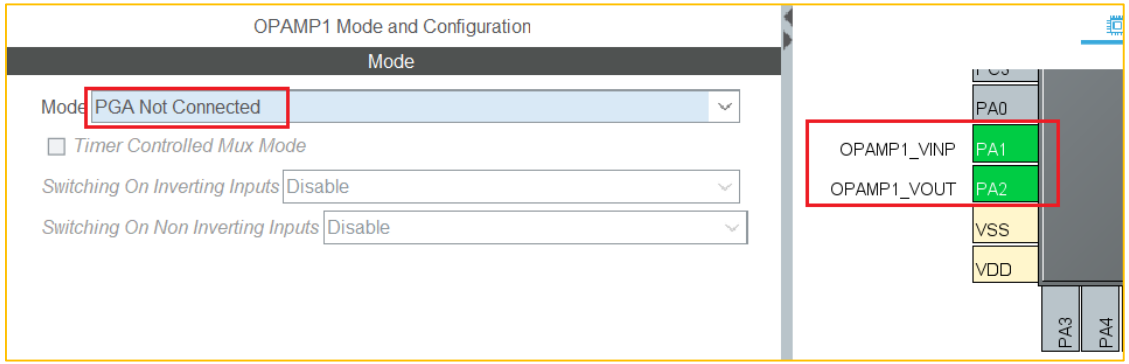


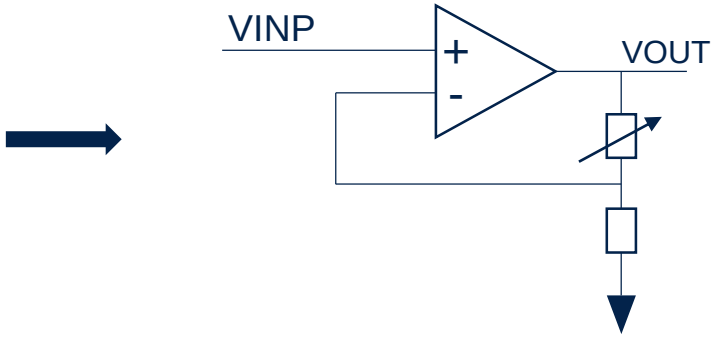


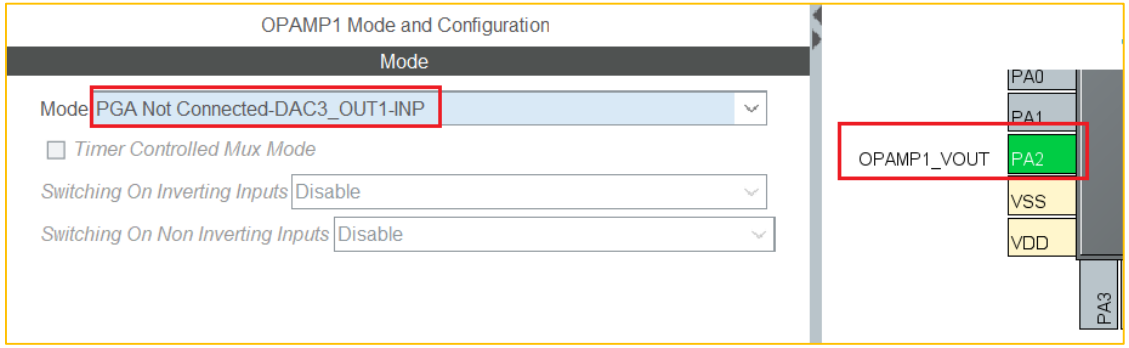
life.augmented

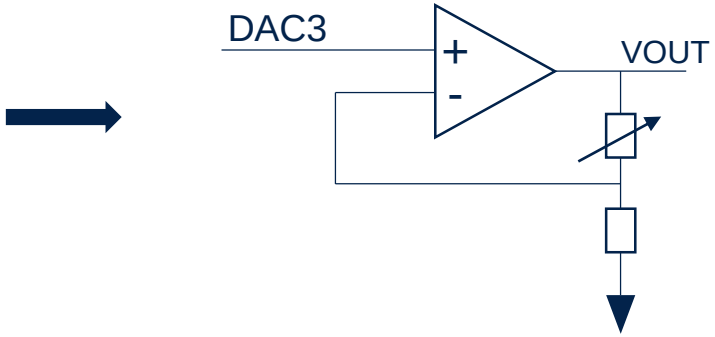
PGA模式与软件配置(1/3)

- PGA模式，无外部VINM模式





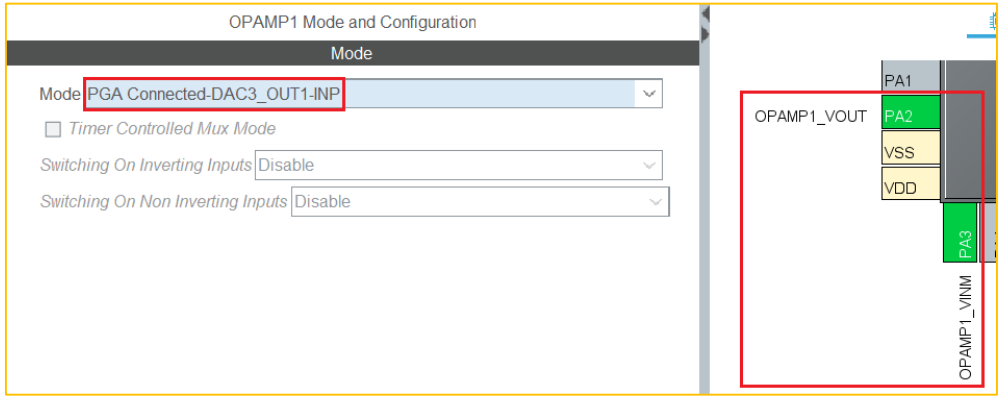


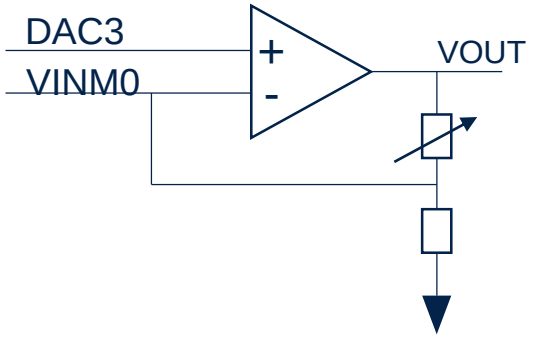


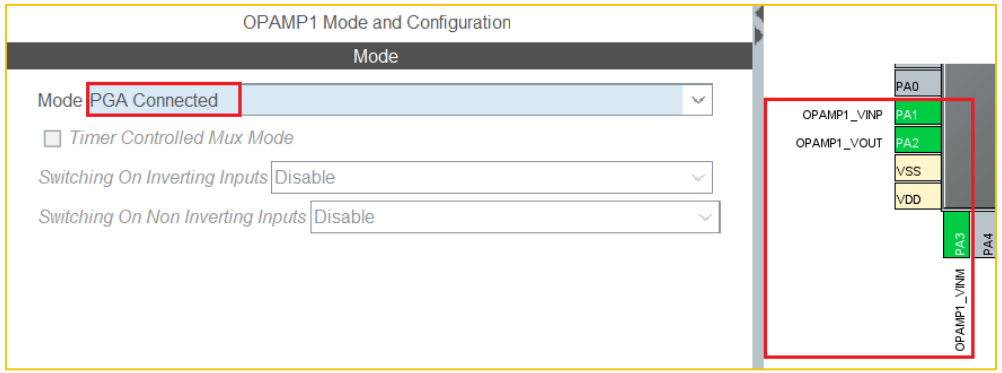
life.augmented

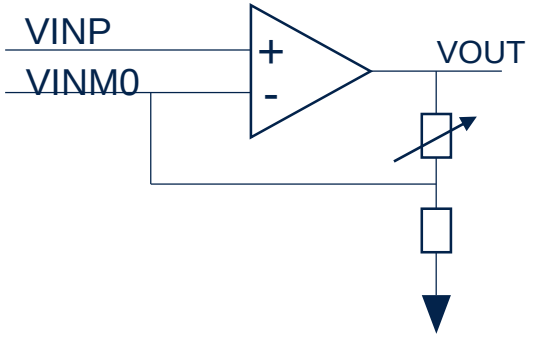
PGA模式与软件配置(2/3)

- PGA滤波外置模式
  - VINM0与VOUT之间可以连接滤波电容



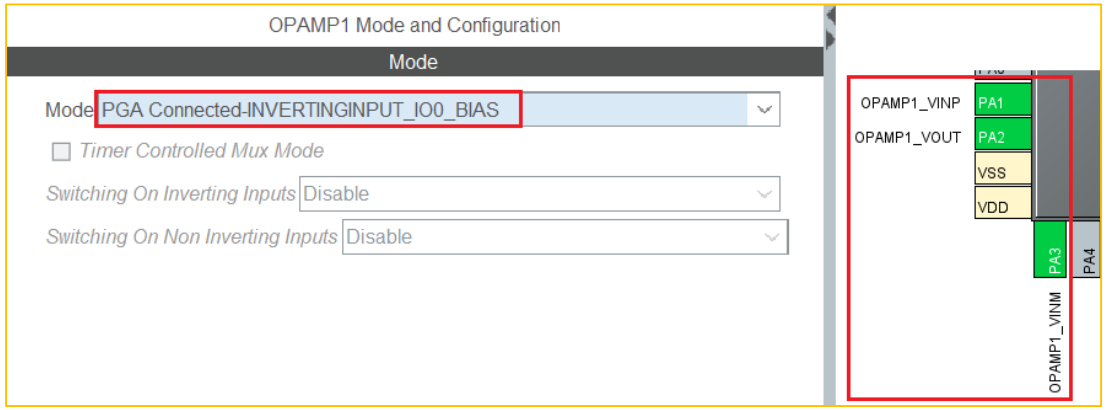


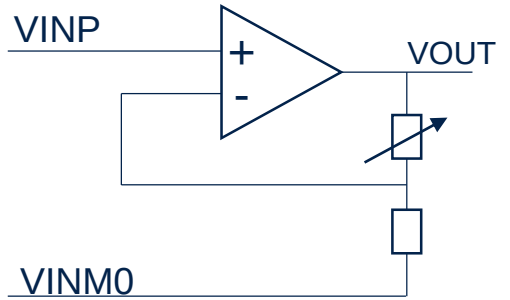




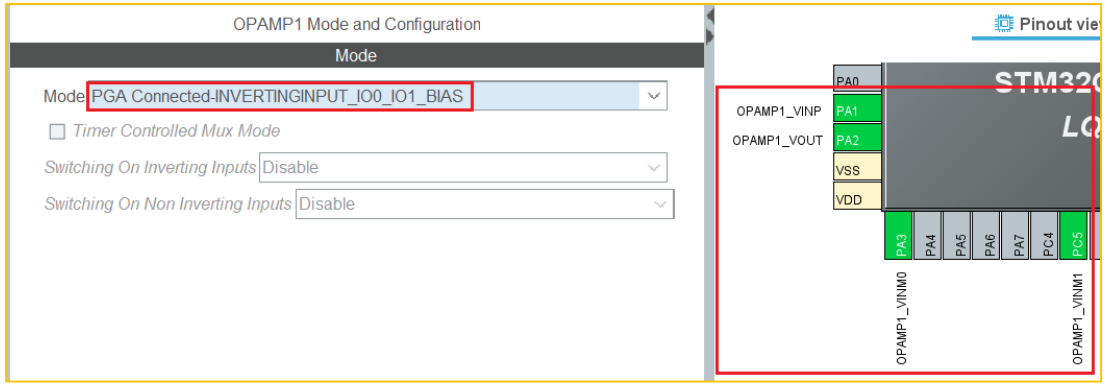
PGA模式与软件配置(3/3)

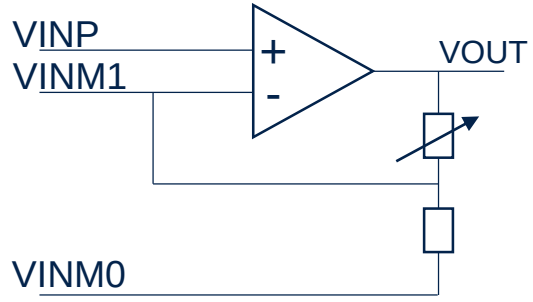
- PGA带偏置模式





- PGA带偏置与滤波模式





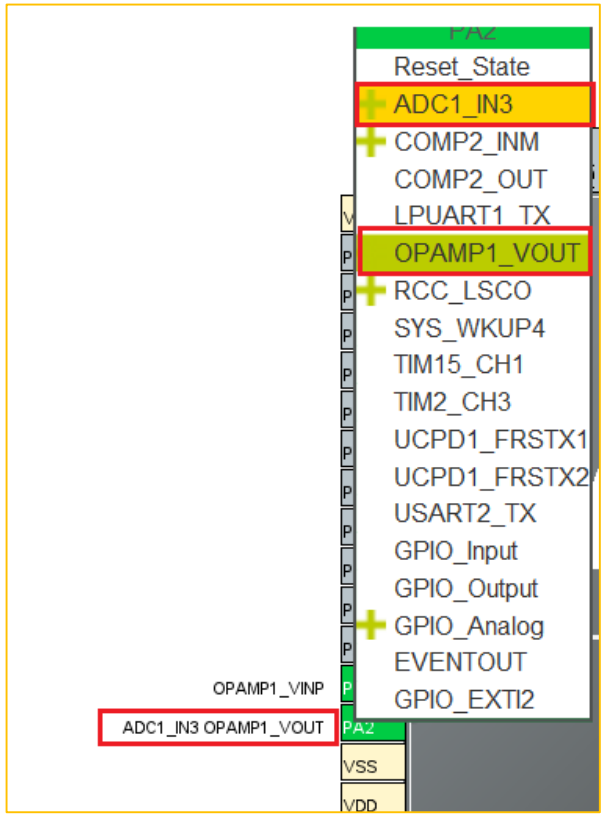
life.augmented

运放连接到内部ADC模块

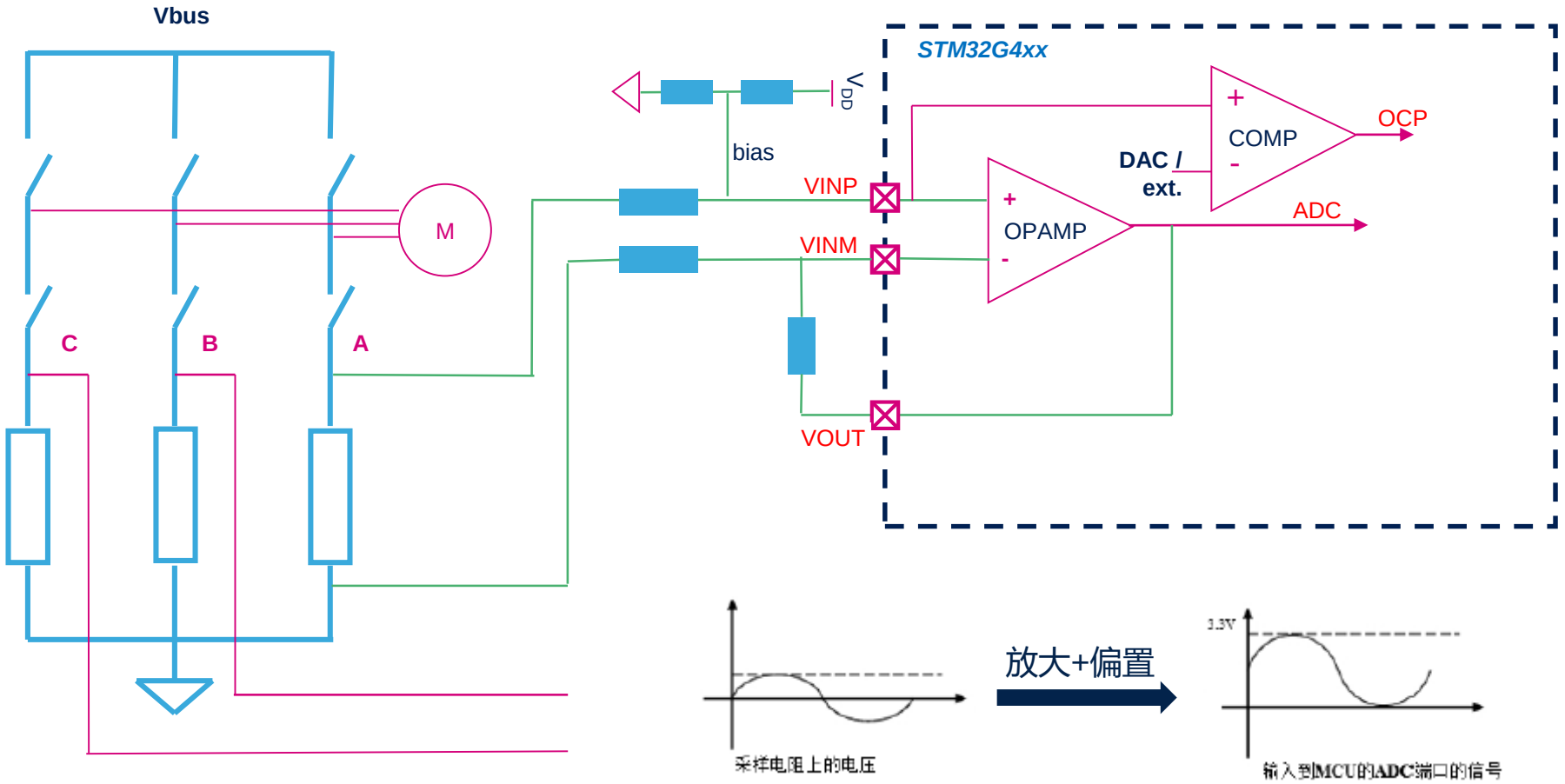
- 方式一：内部连接ADC，无需占用管脚
  - 设定寄存器输出配置位为1：OPAMPx_CSR.OPAINTOEN = 1

	ADC1	ADC2	ADC3	ADC4	ADC5
IN1	PA0	PA0	PB1	PE14	PA8
IN2	PA1	PA1	PE9	PE15	PA9
IN3	PA2	PA6	PE13	PB12	OPAMP5 (int.)
IN4	PA3	PA7	PE7	PB14	TempSensor
IN5	PB14	PC4	PB13	PB15	OPAMP4 (int.)
IN6	PC0	PC0	PE8	PE8	PE8
IN7	PC1	PC1	PD10	PD10	PD10
IN8	PC2	PC2	PD11	PD11	PD11
IN9	PC3	PC3	PD12	PD12	PD12
IN10	PF0	PF1	PD13	PD13	PD13
IN11	PB12	PC5	PD14	PD14	PD14
IN12	PB1	PB2	PB0	PD8	PD8
IN13	OPAMP1 (int.)	PA5	OPAMP3 (int.)	PD9	PD9
IN14	PB11	PB11	PE10	PE10	PE10
IN15	PB0	PB15	PE11	PE11	PE11
IN16	TempSensor	OPAMP2 (int.)	PE12	PE12	PE12
IN17	VBAT/3	PA4	VBAT/3	OPAMP6 (int.)	VBAT/3
IN18	VREFINT	OPAMP3 (int.)	VREFINT	VREFINT	VREFINT

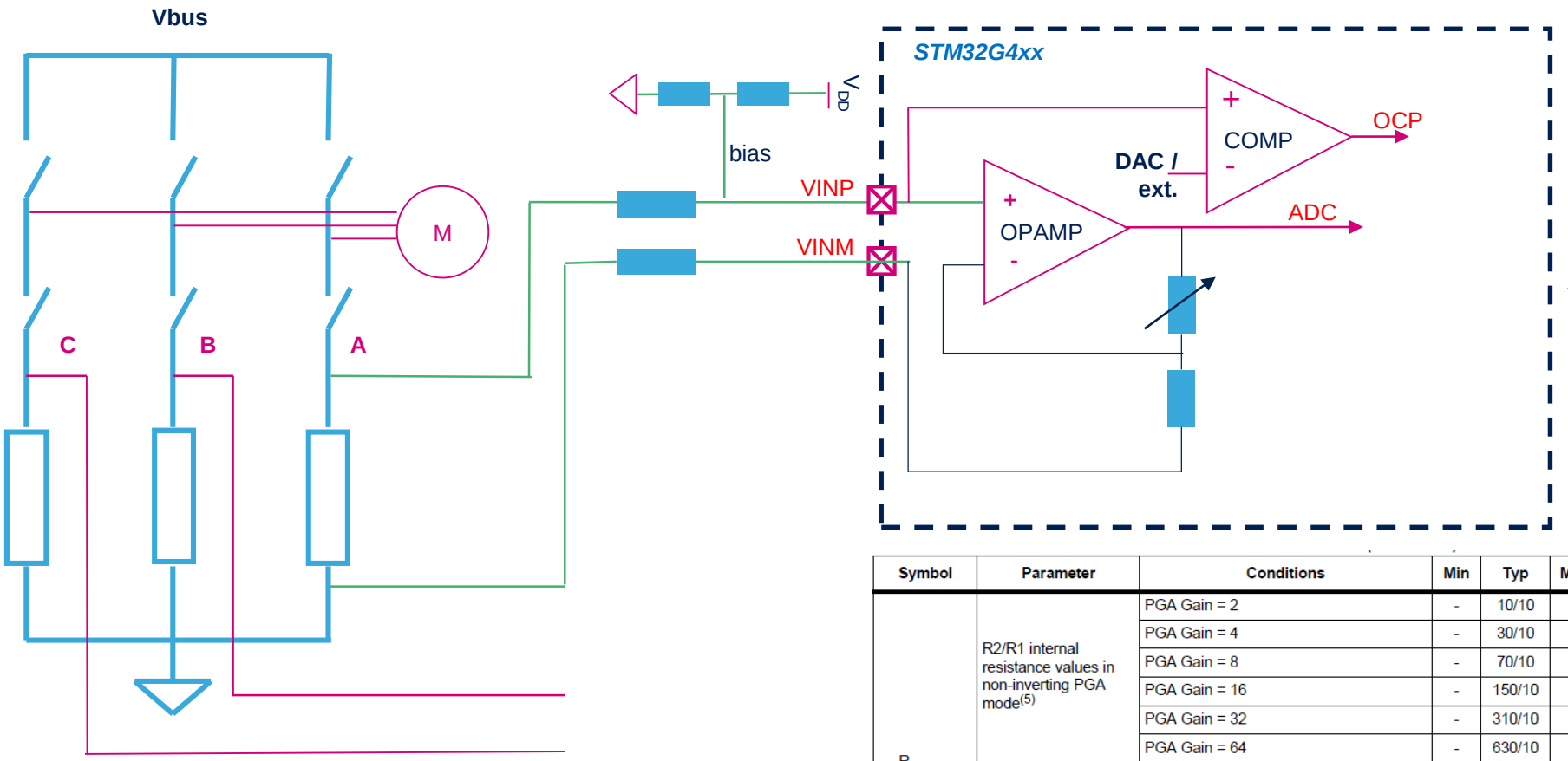
- 方式二：管脚复用连接到ADC
  - 管脚即作为运放输出也作为ADC的输入管脚



STM32G4内部运放实际应用(1/2)



STM32G4内部运放实际应用(2/2)



注意在不同的内部增益配置时不同的阻抗组合！

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
$R_{network}$	R2/R1 internal resistance values in non-inverting PGA mode(5)	PGA Gain = 2	-	10/10	-	kΩ/kΩ
		PGA Gain = 4	-	30/10	-	
		PGA Gain = 8	-	70/10	-	
		PGA Gain = 16	-	150/10	-	
		PGA Gain = 32	-	310/10	-	
		PGA Gain = 64	-	630/10	-	
	R2/R1 internal resistance values in inverting PGA mode(5)	PGA Gain = -1	-	10/10	-	
		PGA Gain = -3	-	30/10	-	
		PGA Gain = -7	-	70/10	-	
		PGA Gain = -15	-	150/10	-	
		PGA Gain = -31	-	310/10	-	
		PGA Gain = -63	-	630/10	-	
Delta R	Resistance variation (R1 or R2)	-	-15	-	+15	%

life.augmented

STM32G4内部运放补充

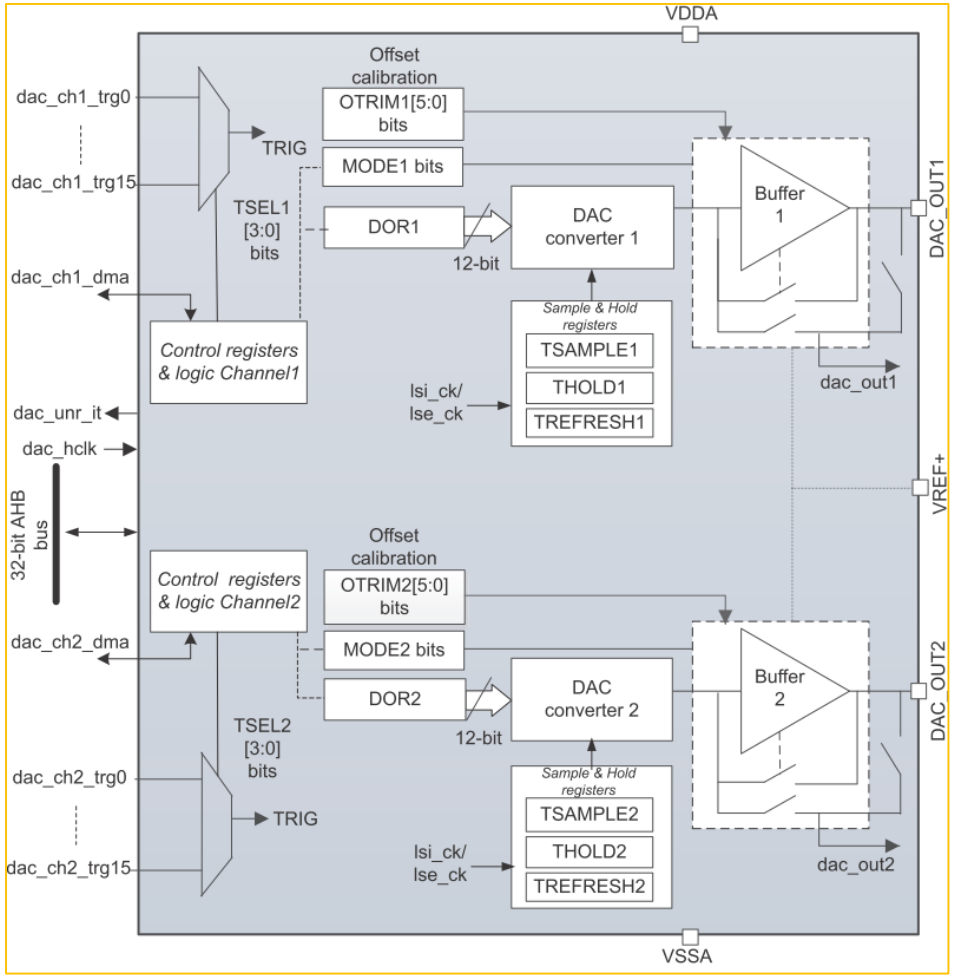
- 可选择模式很多，可以根据需要进行配置
- 熟练后，可以直接写OPAMPx_CSR寄存器进行配置
- 独立模式下增益以及外部偏置可以外部电路配置

高性能DAC

life.augmented

DAC总览

- 多达4个DAC模块，12-bit精度
- 多达7个通道:
  - 3个外部通道(外部管脚):
  - 4个内部通道(快速):
- Dual DAC 通道特征
  - 单独转换或是同时转换
- DMA双数传输能力 – 减少总线负荷
- VREF+ 可由内部或是外部提供(on VREF+ pin)
- 波形发生器 (随机噪声, 三角波,锯齿波)
- 多种触发源(软件，定时器，高精度定时器，外部触发源)
- 无符号与有符号数据格式



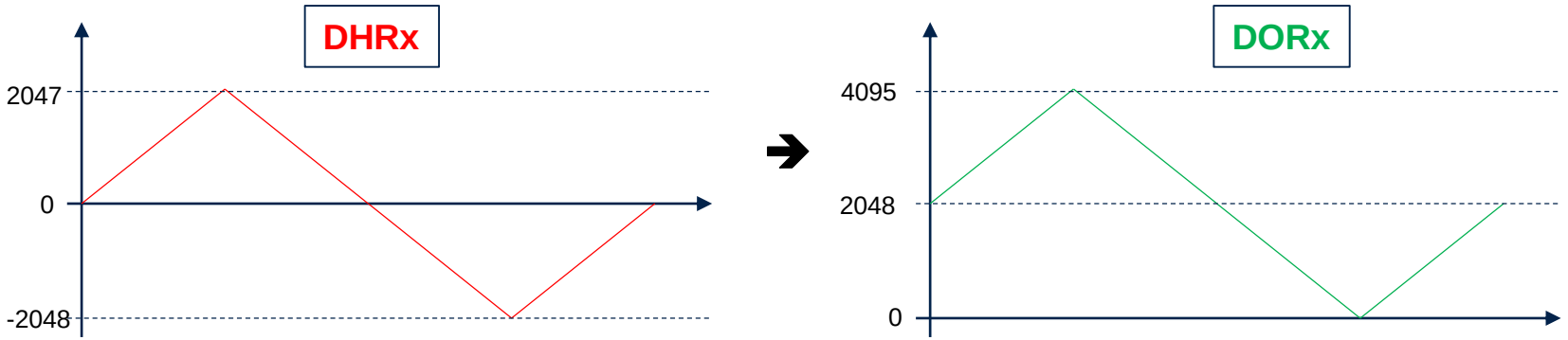
DAC 通道配置

DAC features	DAC1	DAC2	DAC3	DAC4
最大采样速率	1Msps		15Msps	
输出缓冲	x	x		
输出通道	2 (external)	1 (external)	2 (internal)	2 (internal)
IO连接	CH1 on PA4 CH2 on PA5	CH1 on PA6	无直接连接GPIO	

- DAC 速度:
  - 15MHz:
    - 仅限内部通道 (DAC3_OUTx, DAC4_OUTx)
    - 可通过内部运放输出到外部管脚
    - 无缓冲模式
  - 1MHz:
    - 3 个外部通道 (DAC1_OUTx, DAC2_OUT1)
    - 缓冲/无缓冲模式
  - DAC 挂在AHB总线上:
    - 高速，低延时
- 缓冲:
  - 每个缓冲通道都有偏置校正

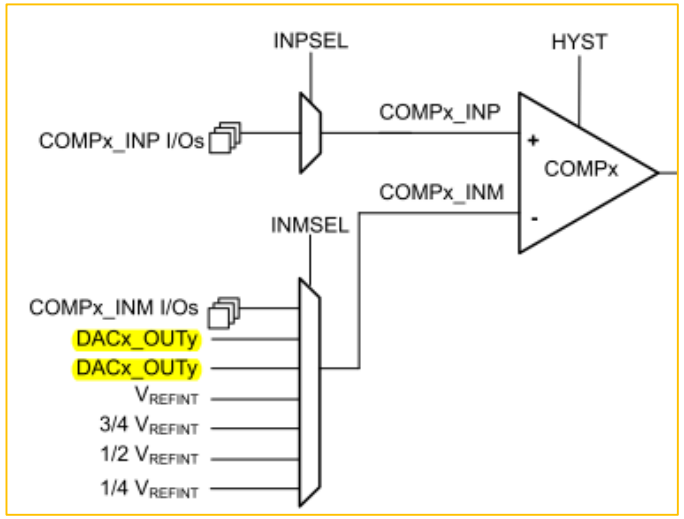
DAC 有符号/无符号数据格式

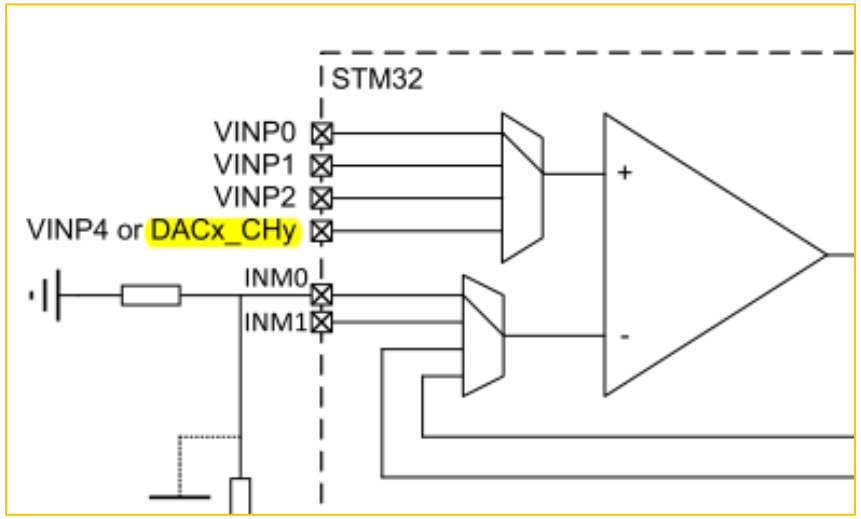
- 无符号数据格式 (SINFORMATx = 0):
  - 标准格式，DAC通道连续输出数据
- 有符号数据格式(SINFORMATx = 1):
  - 新特性: DAC有符号输入格式(DHRx register)
  - 当数据从DHRx传输到DORx，MSB位取反:
    - Example: 0x000 → 0x800, 0x7FF → 0xFFF, 0x800 → 0x000, 0xFFF → 0x7FF
  - 所有DAC精度都支持有符号格式(8, 12)



DAC内部输出

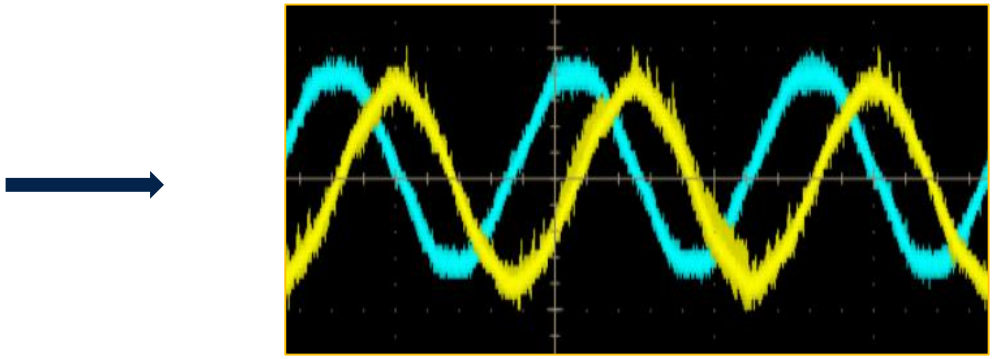
- 比较器的负端输入INM
- 运放的正端输入VINP

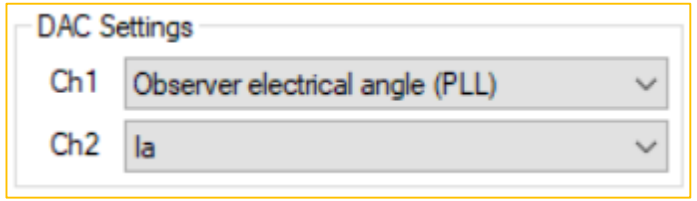


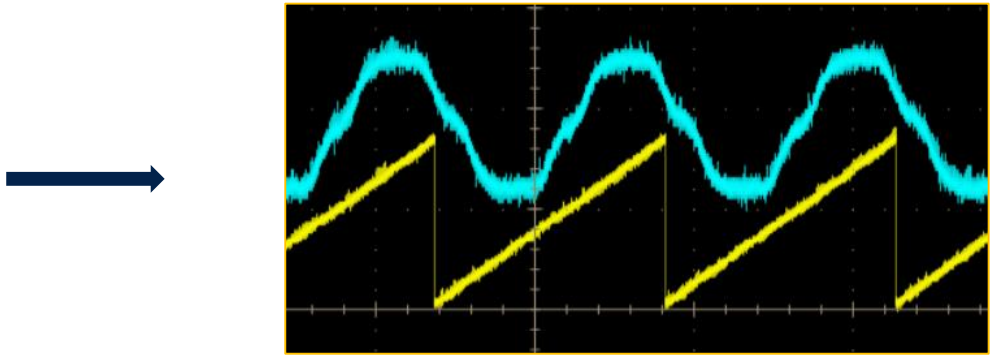


电机控制使用DAC调试

- 配置DAC装载DR(DOR或DHR)为每个PWM波更新一次
- 将需要输出到DAC端口的变量变为12-bit数据
- 实际操作可以直接更新寄存器



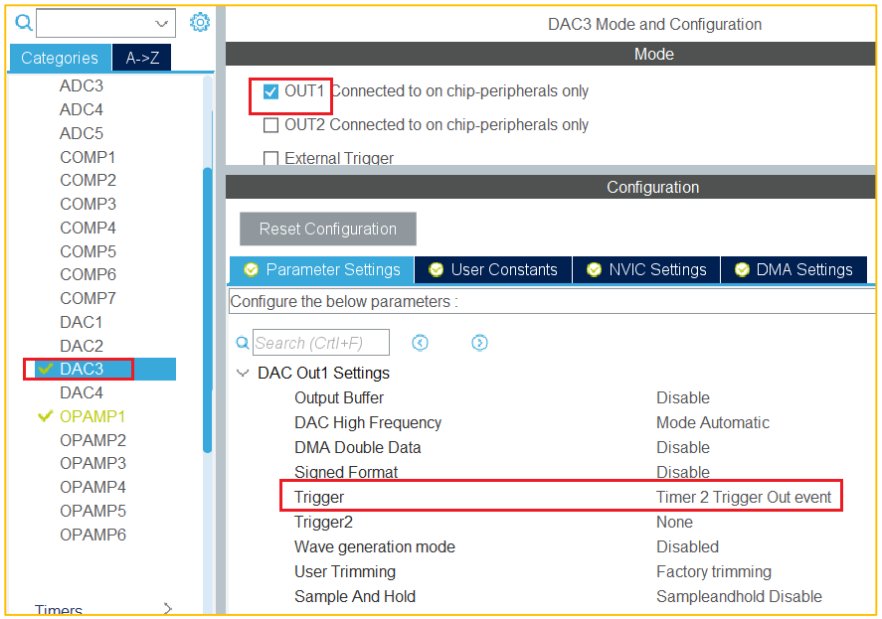


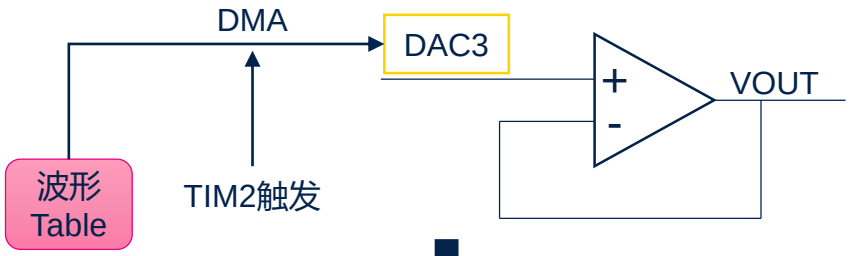


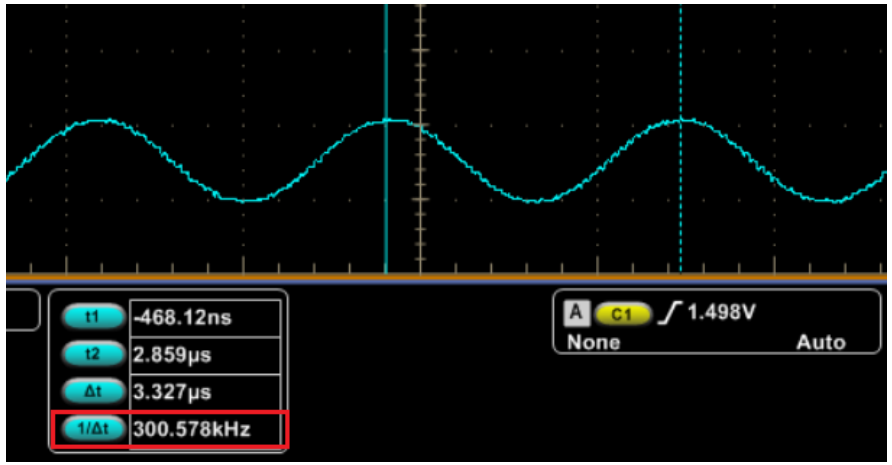
life.augmented

DAC输出高频任意波形

- 举例：输出300KHz，采样40点的正弦波
  - ✓ TIM2频率：170MHz/14
  - ✓ DAC3更新速度：170MHz/14 = 12Msps
  - ✓ 波形频率：12MHz/40 = 300KHz







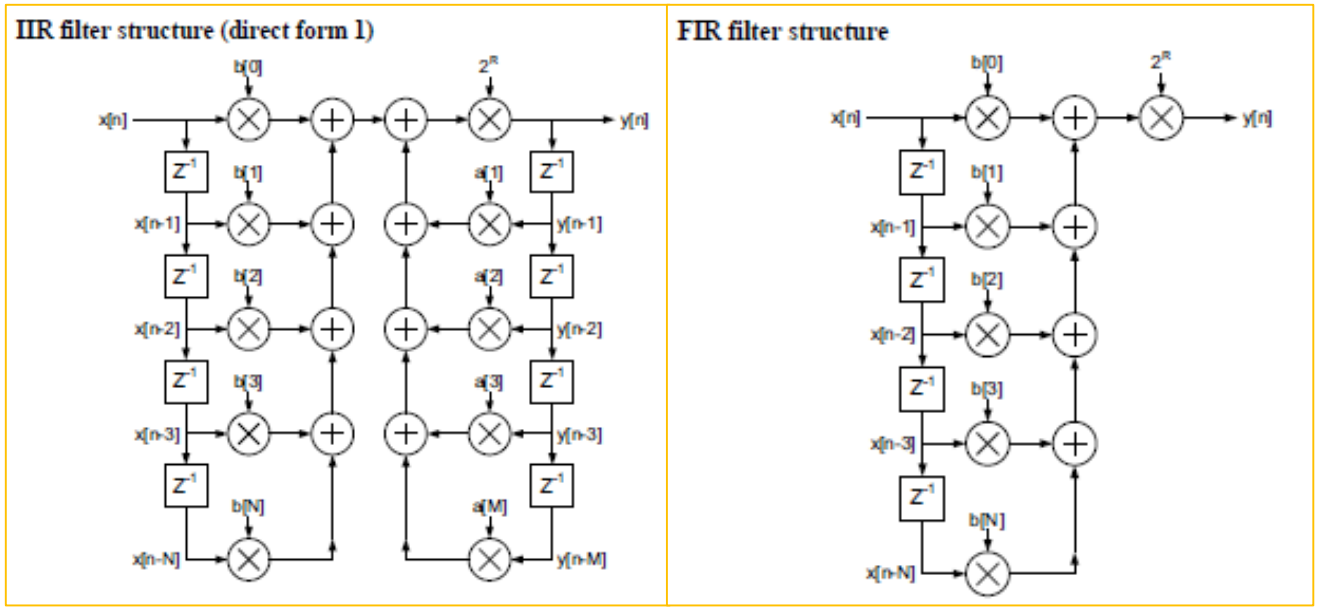
滤波算法加速器（FMAC）

life.augmented

总览

- FMAC(Filter Math Accelerator)用于计算滤波算法
  - 计算有限冲击响应数字滤波器（FIR--Finite Impulse Response)
  - 计算无限冲击响应数字滤波器（IIR --Infinite Impulse Response)
- 低通, 高通, 带通, 带阻, 补偿器等都可以实现
- 速度与ARM math库中的软件计算相当(ARM-M4内核的MAC有助于滤波运算)
- 可以使用DMA释放CPU，达到最优环路效果

滤波方程



- 滤波方程

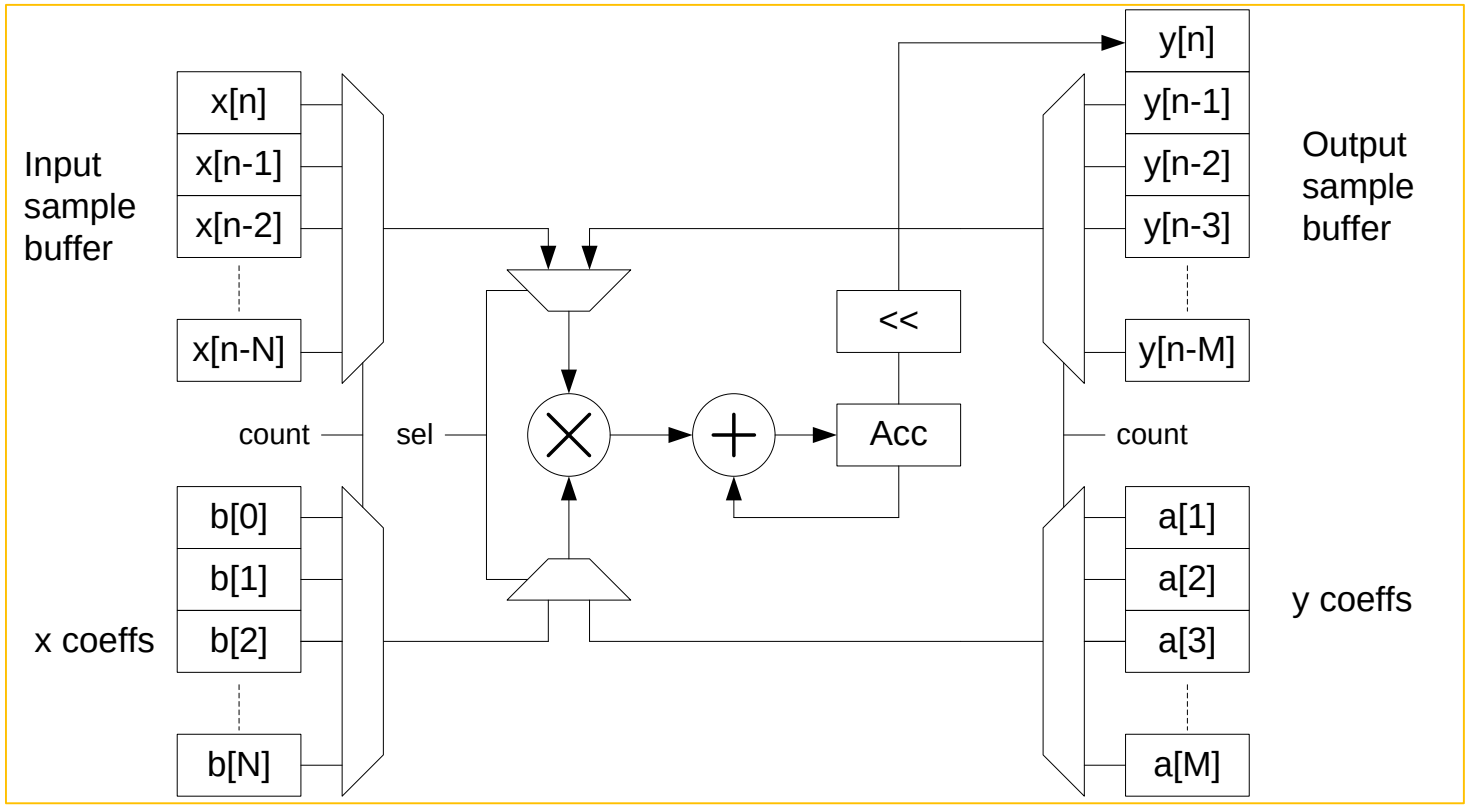
$$y[n] = \sum_{k=0}^{N} b[k]x[n-k] + \sum_{j=1}^{M} a[j]y[n-j]$$

  - x[n] : 第n个输入数据(nth input sample)
  - y[n] : 第n个输出数据(nth output sample)
  - b[k] : 第k个前馈系数(kth feed-forward coefficient)
  - a[j] : 第j个反馈系数(jth feed-back coefficient)

- 有限冲击响应数字滤波器(FIR)是输入采样数据流x[n]与一组系数b[k]的卷积
- 无线冲击响应数字滤波器(IIR)由FIR以及FIR的输出y[n]与系数a[j]的卷积组成

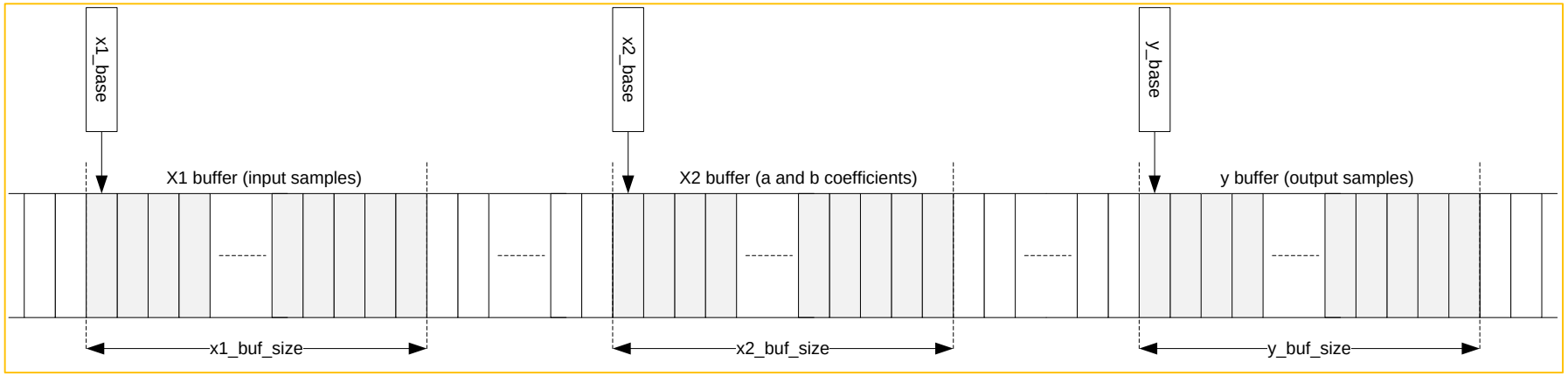
单乘累加架构

- 每个输出都需要N+M+1次乘累加操作
  - 为了降低成本，采用一个乘累加器重复使用



Buffer 配置

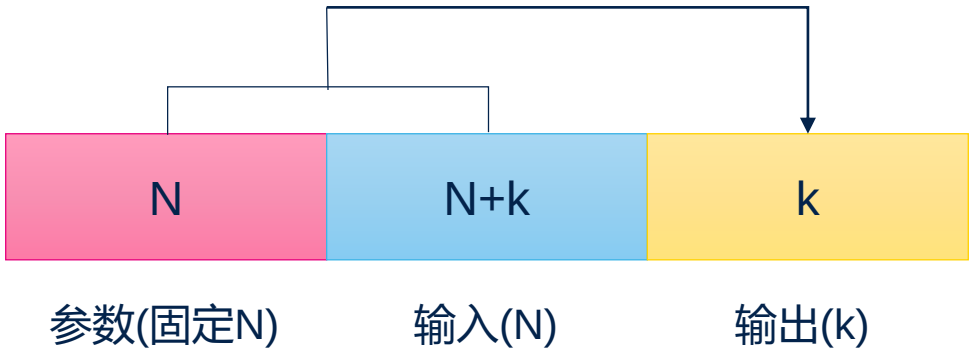
- FMAC本身含有256x16-bit 存储空间用做buffer
- 3个buffer需配置
  - X1 – 输入值(循环)
  - X2 – 滤波参数(固定)
  - Y – 输出值(循环)
- 每个buffer的基址与大小都可配置
  - FMAC_X1BUFCFG，FMAC_X2BUFCFG，FMAC_YBUFCFG



life.augmented

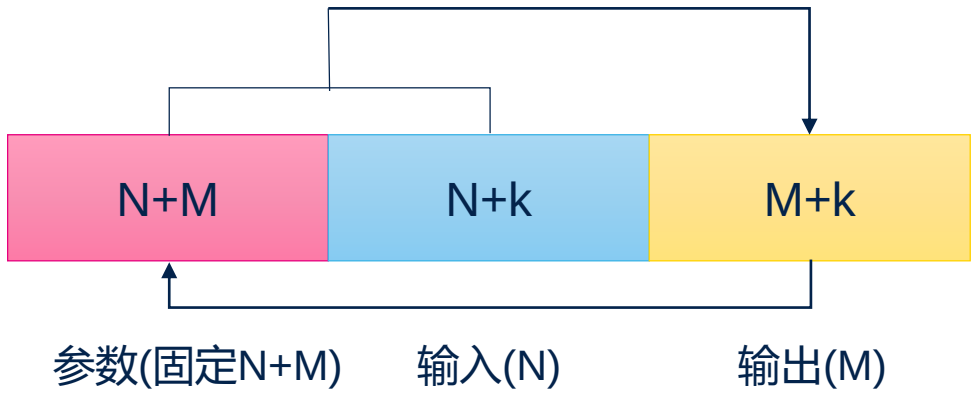
存储空间需求--FIR

- FIR
  - N个系数与N个输入值计算产生1个输出值
  - 每个输入值被使用N次
  - 为优化吞吐率，输入buffer大小应配置为N + k，一个DMA请求可装载k个暂不参与运算的输入值
  - 输出buffer大小应配置为k，以匹配DMA传输长度
  - 总的空间需求为: 2(N + k) 且2(N + k) < 256



存储空间需求--IIR

- IIR
  - N个前馈系数(B:feed-forward coefficients)与M个反馈系数(A: feed-back coefficients)，且(M < N)
  - N个输入值与M个输出值计算产生1个输出值
  - DMA传输长度为k
  - 总的存储空间需求为: 2(N + M + k)且 2(N + M + k) < 256



life.augmented

FMAC Vs 软件计算

- Single stage bi-quad/2p2z (IIR with N=3, M=2)
  - arm_biquad_cascade_df1_fast_q15():
    - 2048 samples processed in 22730 cycles ⇒ 12 cycles/sample
  - FMAC (using DMA in, DMA out):
    - 2048 samples processed in 27310 cycles ⇒ 14 cycles/sample
- 51-tap FIR (N=51)
  - arm_fir_fast_q15():
    - 2048 samples processed in 187418 cycles ⇒ 92 clock cycles/sample
  - FMAC (using DMA in, DMA out):
    - 2048 samples processed in 212683 cycles ⇒ 104 clock cycles/sample

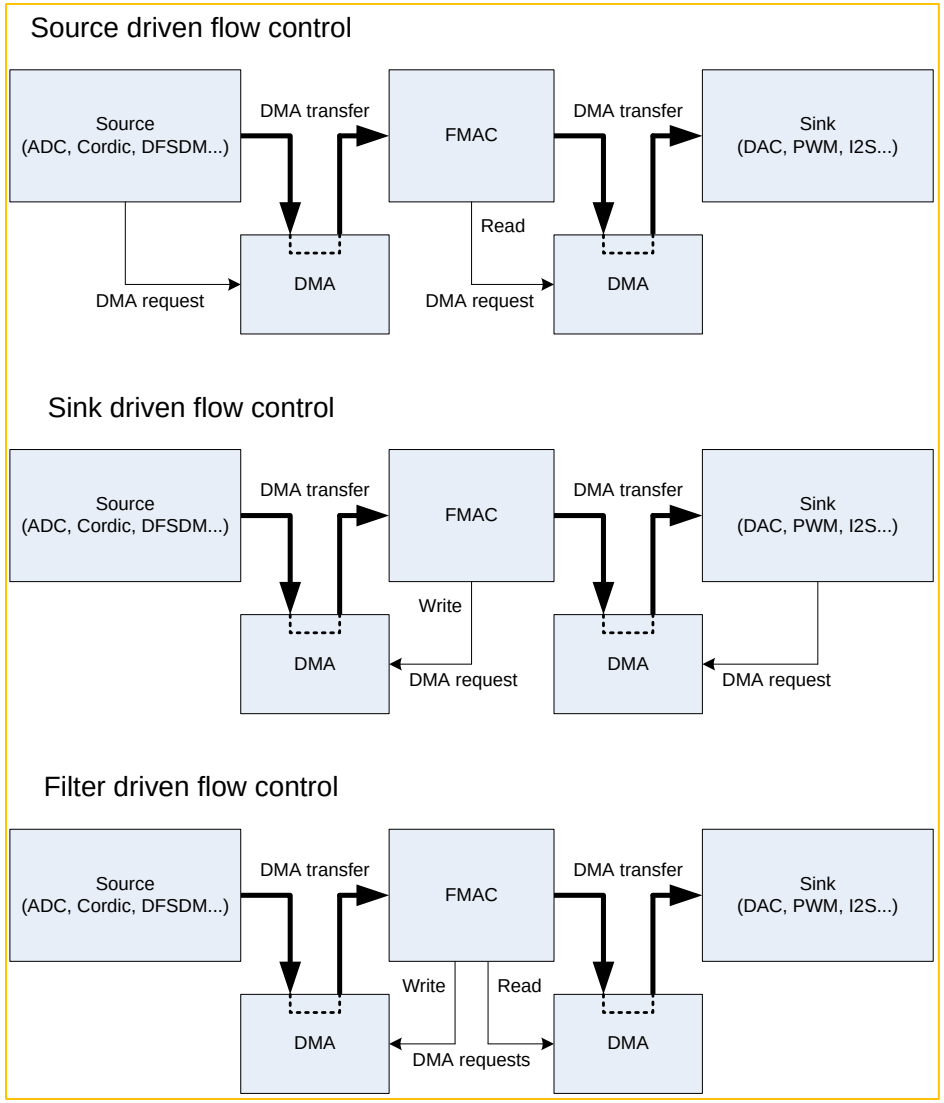
- 软件方式比FMAC快15%，原因是Cortex-M4具备双硬件乘累加器(MAC)
  - 但是对于高速采样/长滤波， Software 方式CPU需要花费大量时间来执行滤波任务
  - 使用FMAC+DMA方式，无需CPU参与，CPU可执行其他任务

工作流控制

源驱动(Source driven)

端驱动(Sink driven)

滤波器驱动(Filter driven)



数据格式

- FMAC采用定点有符号整型格式
  - 输入与输出值数据格式为q1.15
  - q1.15 格式由1bit符号位与15bit小数位(二进制小数位)构成，数据范围为-1 (0x8000) to 1 - 2^{-15} (0x7FFF)
- 累加器为26bit，其中22bit为分数，4bit为有符号整数(q4.22)
  - 乘法器的输出格式为q2.30，被截断为q2.22，然后与累加器的低位对齐相加
  - 若过程结果超出[-8， 7.99999976]的范围，SAT标志将置位，触发中断(if SATIEN is set)
- 累加器输出结果的增益可调整
  - 从 0dB 到 42dB（0~2^7），步长 6dB

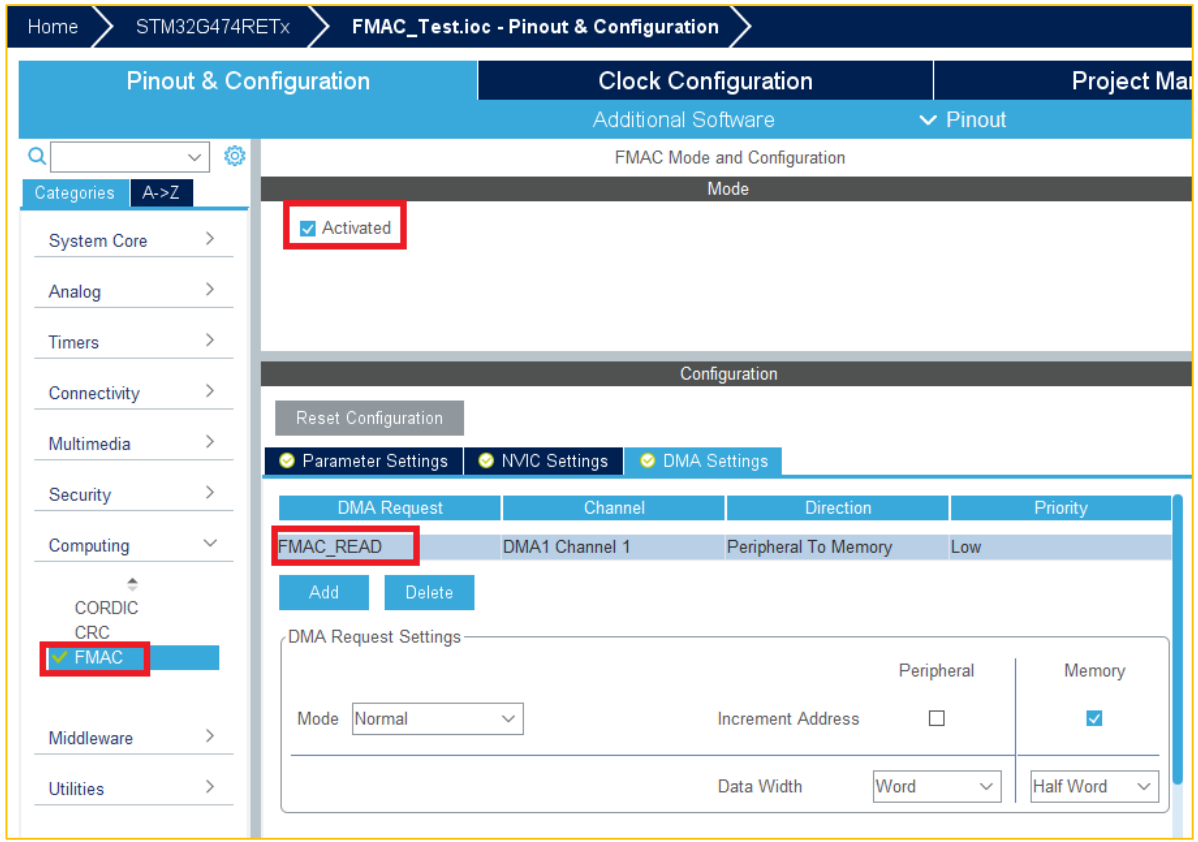
FMAC配置寄存器

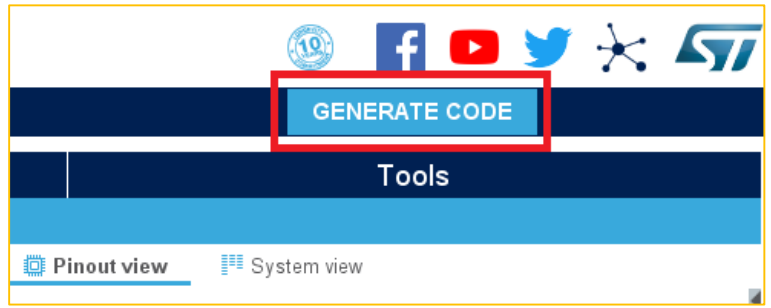
- 对于STM3G4的FMAC是对下面的寄存器的操作

寄存器名称	寄存器说明	附加说明
FMAC_X1BUFCFG	X1 buffer 寄存器	输入数据
FMAC_X2BUFCFG	X2 buffer 寄存器	滤波参数
FMAC_YBUFCFG	Y buffer 寄存器	输出数据
FMAC_PARAM	FMAC参数寄存器	可控制何种数据的装载
FMAC_CR	FMAC控制寄存器	
FMAC_SR	FMAC状态寄存器	可读取状态
FMAC_WDATA	FMAC写入寄存器	
FMAC_RDATA	FMAC读取寄存器	

使用FMAC举例(1/10)

- 使能FMAC运算单元，并生成工程文件
- 配置FMAC DMA

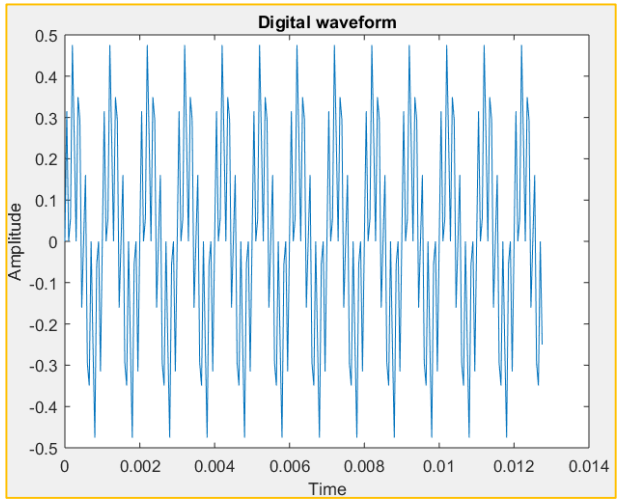




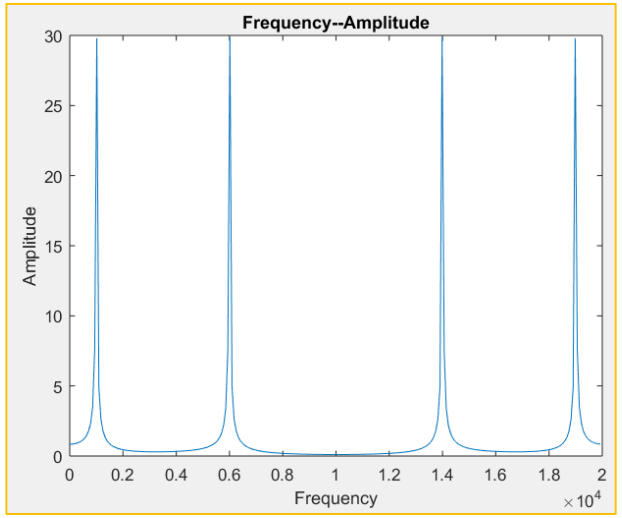
使用FMAC举例(2/10)

- 需要根据需求设计数字滤波器
  - 假定一个1KHz的正弦波叠加一个6KHz中正弦波
  - 使用20KHz采样频率进行波形采样，后得到如下数字量
  - 类似状况就向ADC采样一般，为IIR或者FIR输入数字量

```
/* Array of input values in Q1.15 format */
static int16_t aInputValues[100] =
{
    0,10323,0,1812,15582,8192,0,11443,9630,-5260,
    0,5260,-9630,-11443,0,-8192,-15582,-1812,0,-10323,
    0,10323,0,1812,15582,8192,0,11443,9630,-5260,
    0,5260,-9630,-11443,0,-8192,-15582,-1812,0,-10323,
    0,10323,0,1812,15582,8192,0,11443,9630,-5260,
    0,5260,-9630,-11443,0,-8192,-15582,-1812,0,-10323,
    0,10323,0,1812,15582,8192,0,11443,9630,-5260,
    0,5260,-9630,-11443,0,-8192,-15582,-1812,0,-10323,
    0,10323,0,1812,15582,8192,0,11443,9630,-5260,
    0,5260,-9630,-11443,0,-8192,-15582,-1812,0,-10323
};
```



1KHz 正弦波
+6KHz 正弦波

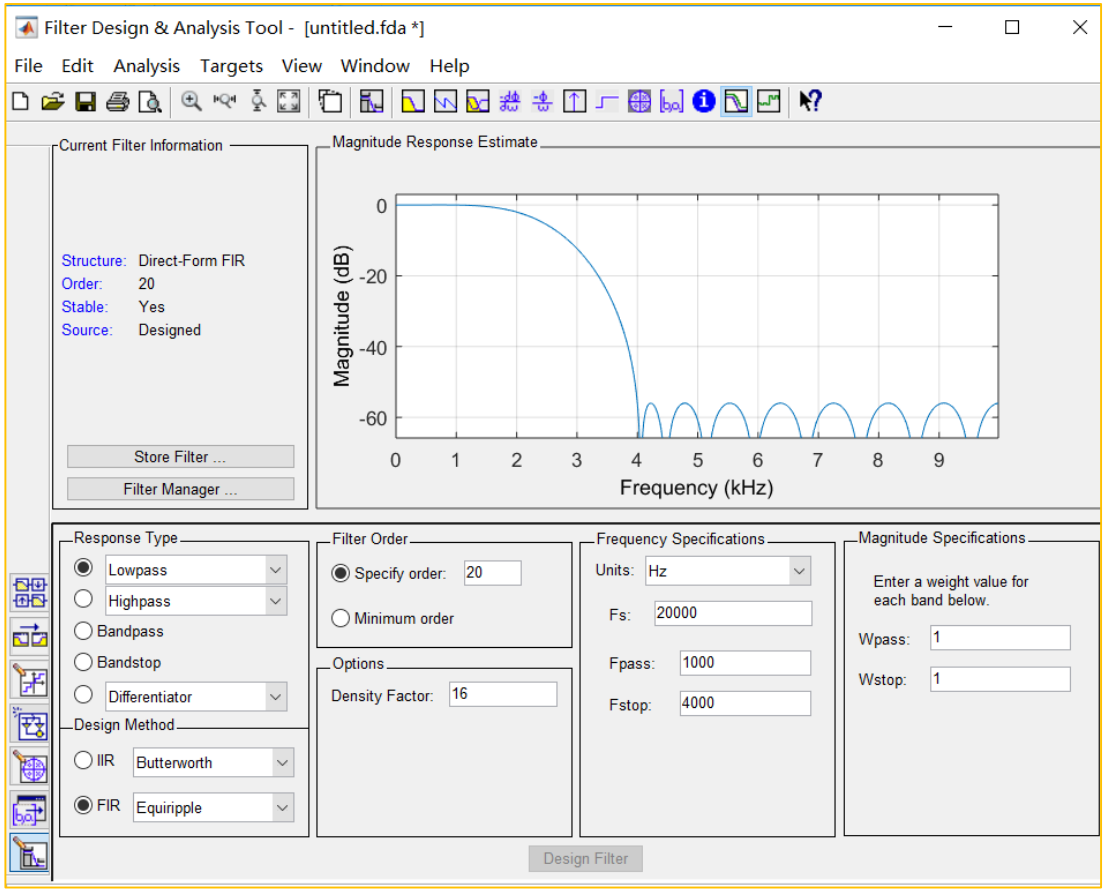


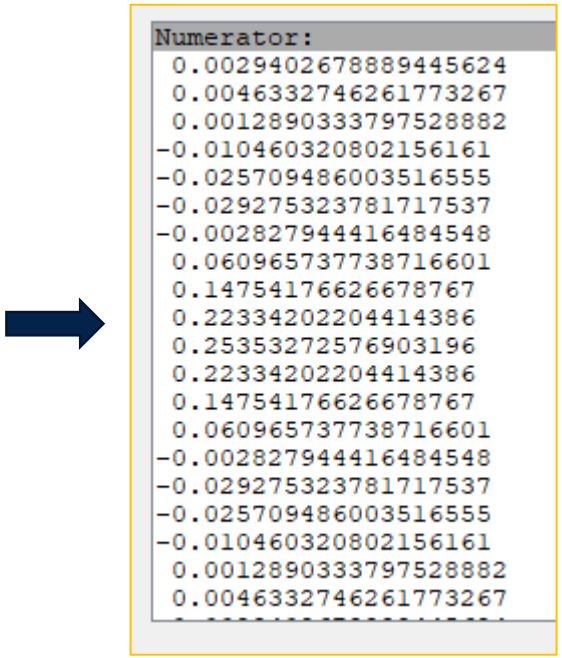
信号FFT波形

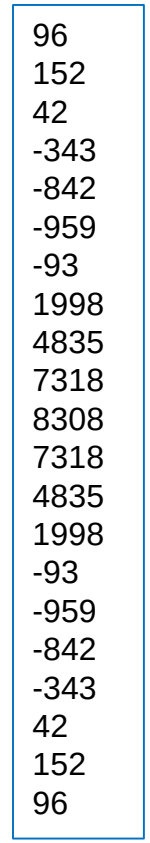
life.augmented

使用FMAC举例(3/10)

- 得到设计FIR滤波器参数
  - 例如设计FIR滤波器1KHz通带，截止4KHz，Butterworth低通滤波器
  - 滤波器软件的系数乘以32768，变为Q15格式，作为FMAC的B系数



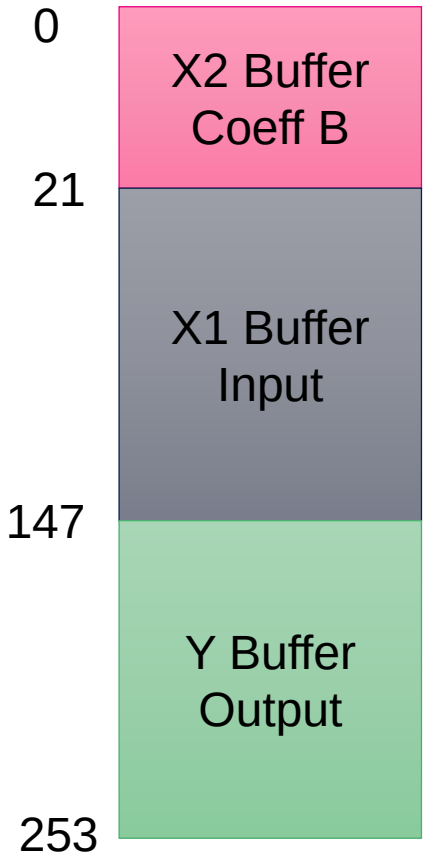




使用FMAC举例(4/10)

- 分配FMAC Buffer地址以及大小
  - 本例子中输入输出数组都为100，系数数组为21
  - N = 21, k = 105(实际上用100就可以)

FIR滤波 $y[n] = \sum_{k=0}^{N} b[k]x[n-k]$



life.augmented

使用FMAC举例(5/10)

- 1.初始化FMAC的FIR功能，并初始化buffer
- 2.输入数组数据装载
- 3.执行FIR滤波，数据输出到数组

1

```
FMAC_FilterConfigTypeDef sFmacConfig;
sFmacConfig.CoeffBaseAddress  = 0;
sFmacConfig.CoeffBufferSize   = FIR_COEFF_B_SIZE;
……
……
HAL_FMAC_FilterConfig(&hfmac, &sFmacConfig);
```

2

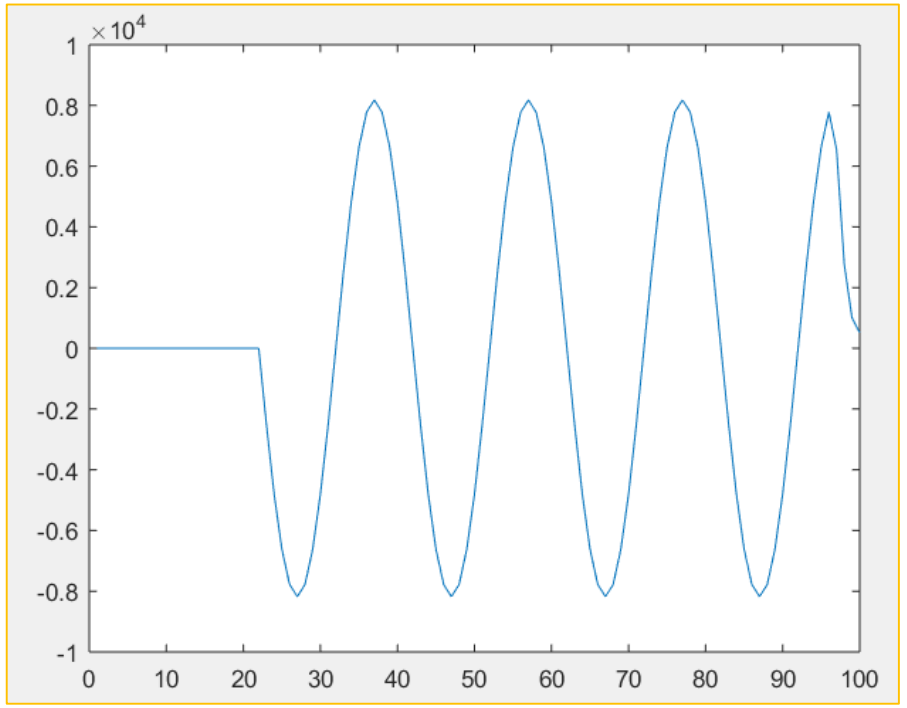
```
HAL_FMAC_FilterPreload(&hfmac, aInputValues, FIR_COEFF_B_SIZE + FIR_D1,
                                    aOutputDataToPreload, FIR_COEFF_A_SIZE +FIR_COEFF_B_SIZE)
```

3

```
HAL_FMAC_FilterStart(&hfmac, aCalculatedFilteredData, &FIR_OutputSize)
```

使用FMAC举例(6/10)

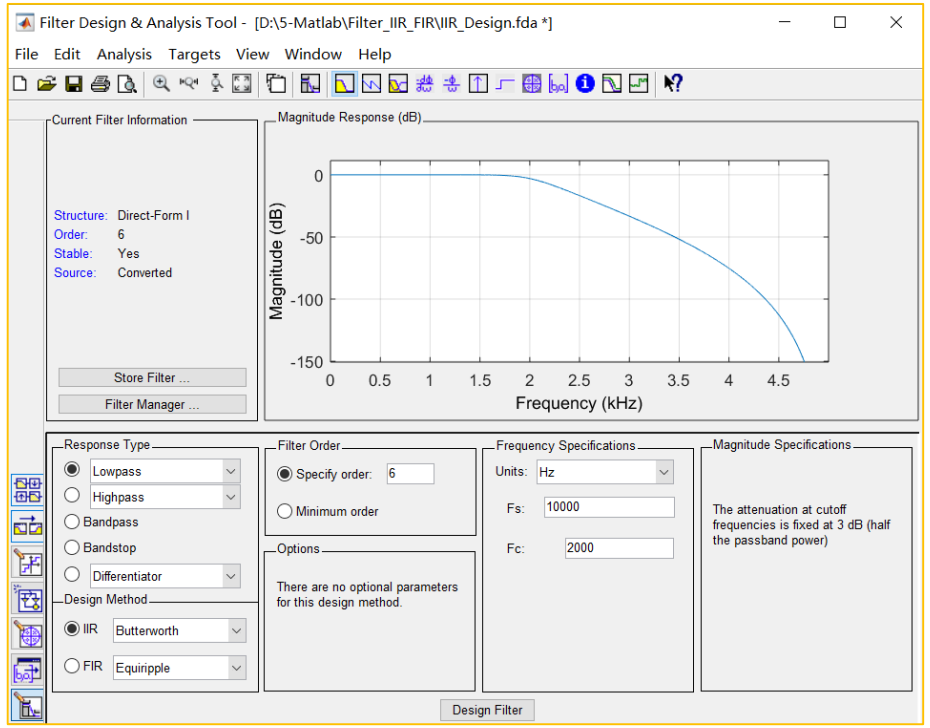
- 验证结果
  - 可以和预先计算后的数组进行比对
  - 可以将数据输出查看波形



life.augmented

使用FMAC举例(7/10)

- IIR滤波器设计，得到系数
  - 例如设计IIR滤波器Fc=2KHz，Butterworth低通滤波器
  - Numerator得到的系数乘以32768，变为Q15格式，作为FMAC的B系数
  - Denominator系数先变为[-1,+1]，然后Q15，再做数据取反作为FMAC的A系数




```
Numerator:
0.010312874762664407
0.061877248575986442
0.15469312143996611
0.20625749525328813
0.15469312143996611
0.061877248575986442
0.010312874762664407
```

Q15 →

338
2028
5069
6759
5069
2028
338

```
Denominator:
 1
-1.1876006801756154
 1.3052133492885514
-0.67432752529799889
 0.26346934828013846
-0.051753033879641426
 0.0050225265950881587
```

Q15 /Gain →

16384
-19458
21385
-11048
4317
-848
82

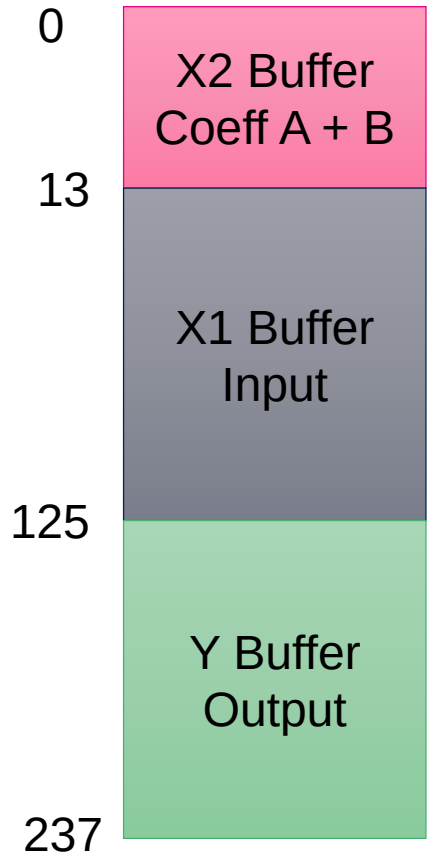
删除第一个系数
其余数据取反 →

19458
-21385
11048
-4317
848
-82

使用FMAC举例(8/10)

- 分配FMAC Buffer地址以及大小
  - 本例子中输入输出数组都为100，6阶滤波IIR
  - N = 7, M = 6，k = 105(实际用100即可)

$$\text{IIR滤波 } y[n] = \sum_{k=0}^{N} b[k]x[n-k] + \sum_{j=1}^{M} a[j]y[n-j]$$



life.augmented

使用FMAC举例(9/10)

- 1.初始化FMAC的IIR功能，并初始化buffer
- 2.输入数组数据
- 3.执行IIR滤波，数据输出到数组

1

```
FMAC_FilterConfigTypeDef sFmacConfig;
sFmacConfig.CoeffBaseAddress  = 0;
sFmacConfig.CoeffBufferSize   = IIR_COEFF_A_SIZE + IIR_COEFF_B_SIZE;
……
……
HAL_FMAC_FilterConfig(&hfmac, &sFmacConfig);
```

2

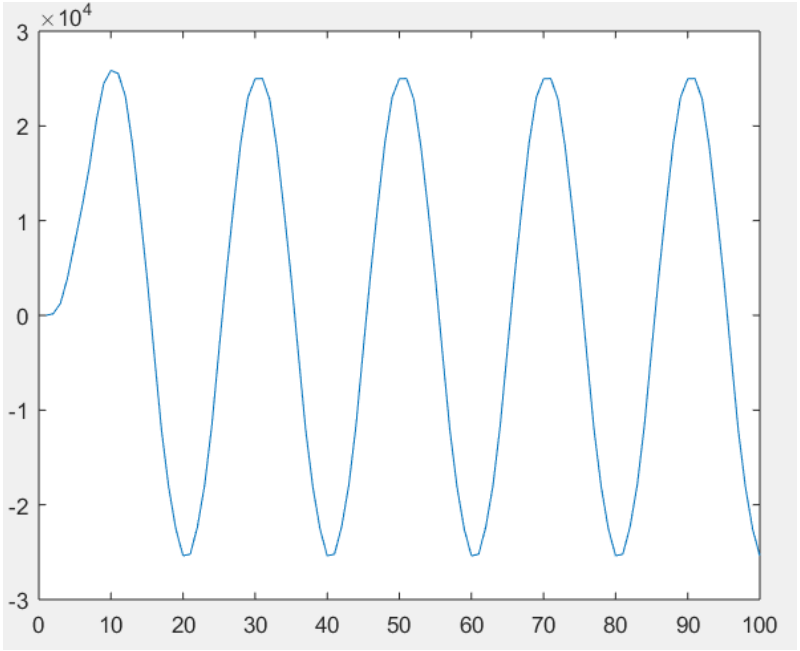
```
HAL_FMAC_FilterPreload(&hfmac, aInputValues, IIR_COEFF_B_SIZE + IIR_D1,
                       aOutputDataToPreload, IIR_COEFF_A_SIZE + IIR_COEFF_B_SIZE);
```

3

```
HAL_FMAC_FilterStart(&hfmac, aCalculatedFilteredData, &IIR_OutputSize);
```

使用FMAC举例(10/10)

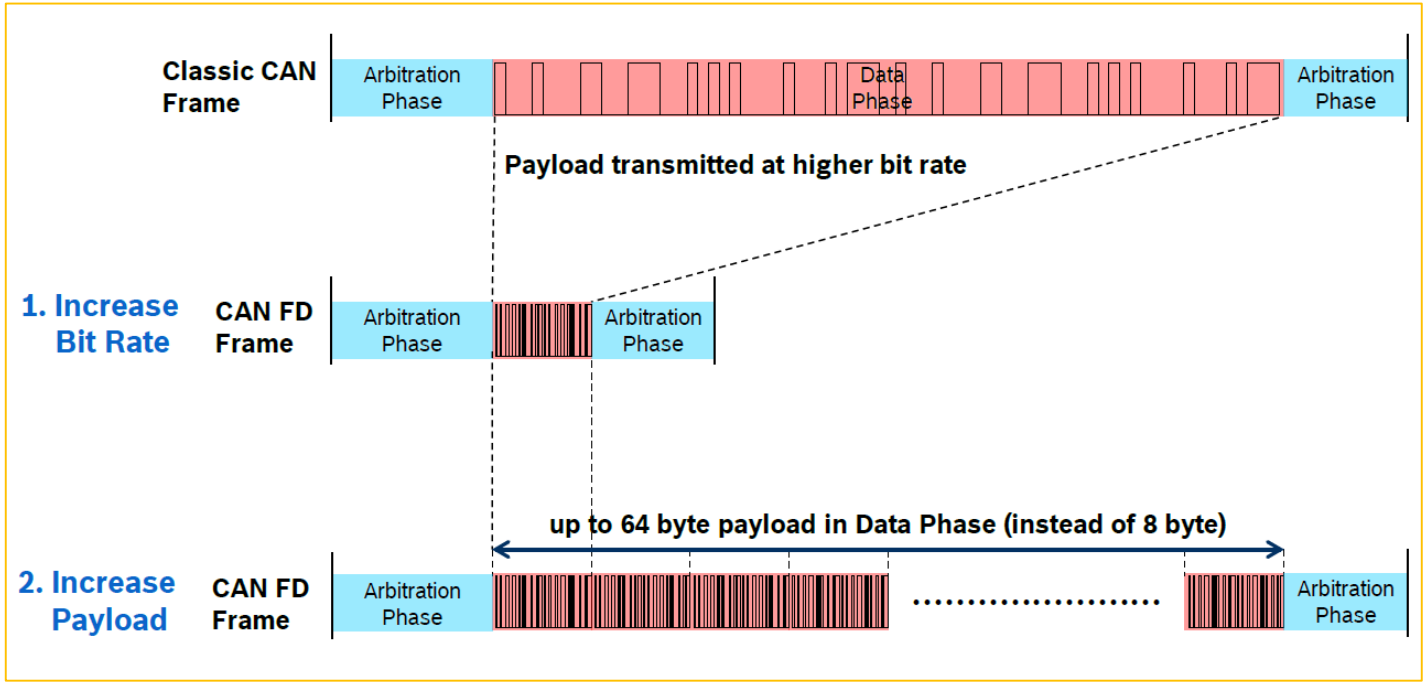
- 验证结果
  - 可以和预先计算后的数组进行比对
  - 也可以将数据输出查看波形



其他特色外设

CAN FD(灵活的数据率)

- CAN FD传输速率可配置
  - 可以减少发送数据，达到高速率
  - 也可以增加发送数据个数，达到有效载荷
  - 兼容传统CAN传输，功能更强



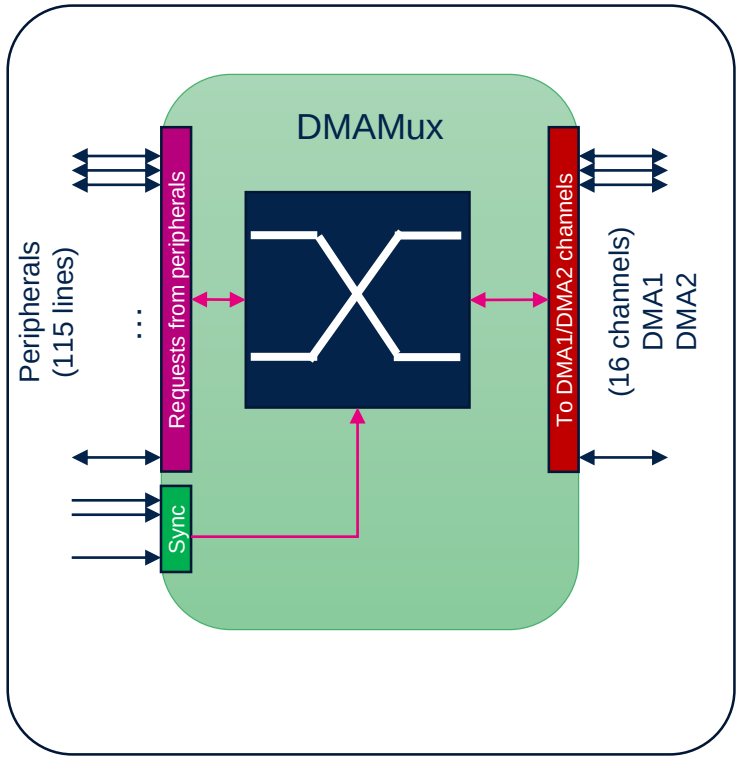
STM32系列单片机中的CAN比较

	bxCAN (STM32F0, F1, F2, F4, F7, L4)	FDCAN (STM32H7)	FDCAN (STM32G4, STM32L5)
ISO 11898-1 Protocol **version**	CAN 2.0A, 2.0B	CAN 2.0A, 2.0B + last version of ISO 11898-1:2015 with CAN FD	
Clock Calibration Unit	No	Yes	No
Time-triggered CAN ISO 11898-4	No	Yes (first instance)	No
Maximum payload size	8 Bytes	64 Bytes	
Maximum Bit rate	1Mbit/s	8Mbit/s	
Transmission	3 mailboxes (1)	up to 32 buffers (FIFO, queue or dedicated buffer modes)	3 buffers with FIFO and queue modes (~ 3 mailboxes) (2)
Tx buffer	No		No
Transmit cancellation	No	Yes	Yes
Tx event FIFO (timestamp)	No	Yes (up to 32 elements)	Yes (up to 3 elements)
Reception	2 FIFOs (3 elements each)	2 FIFOs (up to 64 elements each)	2 FIFOs (3 elements each)
Rx buffer	No	up to 64 message buffers	No
Identifier filters	14 / 28 (3) scalable filter banks Identifier list mode or mask mode	up to 128 11-bit + up to 64 29-bit filters shared between both FDCAN instances	28 11-bit filters + 28 29-bit filters for each FDCAN-lite instance
RAM for TX, RX, filters	512B per instance	10 kB for 2 instances	1kB per instance

life.augmented

DMAMUX

- DMAMUX配置通道的对应外设，数量高达115
- DMAMUX通道0~7对应DMA1通道0~7
- DMAMUX通道8~15对应DMA2通道0~7



	DMAMUX
通道数	115
DMA请求种类	4
触发数量	21
同步数量	21
DMA通道	16 (12 In G431)

DMAMUX	DMA1
CH[0:7]	CH[0:7]

DMAMUX	DMA2
CH[8:15]	CH[0:7]

life.augmented

内部 USB Type-C, USB PD3.0，USB 2.0 device

* UCPD stands for USB Type-C and Power Delivery Interface

- 支持USB Type-C（UC）

- USB Power Delivery interface（PD）

- 可以支持多种最新应用场景
- (验证, 快充, 软件更新等)

总结

- STM32G4的Cordic可以加快环路计算
- STM32G4的FMAC做滤波算法可以释放CPU资源
- 使用出色的片上运放，比较器即省成本又省板材空间
- 特色外设可以有更多设计想象

Thank you

life.augmented